CalculoGradientesCUDA

Guía de desarrollo y apuntes de estudio

Práctica final de Programación Paralela (OpenMP y CUDA)

Objetivo	Calcular de forma exacta y paralela el gradiente de funciones polinómicas de N variables y M monomios, y el Jacobiano de sistemas de K ecuaciones, comparando versiones secuencial, OpenMP y CUDA y documentando el uso de IA generativa.
Autor del proyecto	Antonio Millán Rodríguez (3.º de Ingeniería Informática)
Asignatura	Programación Paralela. Profesor: Baldomero Imbernón Tudela
Fecha	1 de noviembre de 2026
Repositorio	`https://github.com/aantoniomillan/CalculoGradientesCUDA` (rama `master`, commit `2a655c4`)
Hardware de desarrollo	Portátil Lenovo LOQ 15IRH8: Intel Core i5-13420H, NVIDIA GeForce RTX 4050 Laptop, 16 GB de RAM, CachyOS. Detalles de núcleos y GPU en la sección 0.2 **[por verificar]**.
Hardware de entrega	Servidor de prácticas de la asignatura. Características desconocidas: **Supuesto:** se tratan como incógnita hasta medirlas con los comandos de la sección 0.2.
Código de la guía	Todos los fragmentos están en `docs/guia/codigo/` y se han compilado y ejecutado (salvo CUDA, solo compilado) con gcc 13.3, nvcc 12.0, rustc 1.97 y Python 3.13.

Cómo leer este documento

1. Lee la sección 0 (fases) y la 0.1 (diagnóstico): te dicen qué hay que construir, en qué orden y qué existe ya.
2. Usa el índice de temas (sección 0.3) para saber qué estudiar antes de cada fase. Cada tema va de lo básico a lo avanzado; puedes parar cuando tengas lo necesario para la fase en curso y volver después.
3. Cada concepto sigue el esquema *Qué es*, *Por qué lo usamos*, *Cómo se usa* y *Alternativas*. Las cajas naranjas marcan trampas habituales.
4. Antes de cada decisión importante, consulta la tabla de decisiones (sección 13); indica qué medir cuando la respuesta depende de datos.
5. Las marcas **[por verificar]** y **Supuesto:** señalan información no comprobada. No la copies a la memoria sin contrastarla.
6. Las cifras de tiempo de esta guía son orientativas: proceden de una máquina en la nube de 2 vCPU sin GPU, no del servidor de prácticas.

Índice

0 Fases del proyecto **3**
- 0.1 Diagnóstico del estado actual . . . 4
- 0.2 Hardware . . . 6
- 0.3 Índice de temas . . . 6

1 Fundamentos matemáticos **7**

2 Datos, formatos y generadores **9**

3 Algoritmo secuencial **12**

4 Rendimiento y medida **15**

5 OpenMP **18**

6 CUDA: fundamentos **21**

7 CUDA: optimización **23**

8 Sistemas de ecuaciones y Jacobiano **26**

9 Validación y benchmarking **28**

10 Herramientas y entorno **29**

11 IA generativa, prompts y memoria **31**

12 Estado del arte **32**

13 Tabla de decisiones **34**

14 Experimentos, riesgos y calendario **35**
- 14.1 Experimentos planificados . . . 35
- 14.2 Riesgos . . . 36
- 14.3 Calendario con puntos de control . . . 36

15 Validación y reproducibilidad **37**

16 Orientación a la entrega académica **38**

Referencias **38**

0 Fases del proyecto

El proyecto se divide en 16 fases (F0 a F15). Cada una produce ficheros concretos y un entregable que se puede comprobar con un comando. La estructura de directorios propuesta es:

```
CalculoGradientesCUDA/
  Makefile  README.md  Dockerfile  reproduce.sh
  src/      poli.h poli.c secuencial.c secuencial.cpp omp.c grad.cu jac.c jac.cu
  rust/     Cargo.toml src/main.rs
  gen/      genFuncion.py genSistema.py oraculo.py
  tests/    test.sh casos/
  bench/    bench.sh graficas.py
  resultados/  figuras/
  docs/     formato.md prompts.md memoria/ guia/
```

Fase	Qué hacer y ficheros	Entregable verificable	Estado
F0	Reorganizar el repositorio: `src/`, `Makefile`, `.gitignore`, `README.md`; renombrar `CUDA.cpp` a `src/grad.cu` y `paralelo.cpp` a `src/omp.c`; fijar identidad git.	`make all` compila sin avisos; `git log` muestra un único autor.	Parcial: 5 ficheros esqueleto (solo comentarios); README y enunciado borrados en el último commit.
F1	Definir el formato de entrada y escribir `gen/genFuncion.py` y `gen/genSistema.py`; documentar en `docs/formato.md`.	Con la misma semilla se generan ficheros idénticos (`sha256sum`).	Parcial: `generador.py` (CSV) en la rama `claude/fervent-bell-1ekfj8`, sin fusionar ni oráculo.
F2	Oráculo exacto `gen/oraculo.py` con SymPy que escribe `ref.txt`.	Para el ejemplo del enunciado escribe `8 15 8`.	Pendiente (prototipo en `docs/guia/codigo/oraculo.py`).
F3	Estructura CSR y parser: `src/poli.h`, `src/poli.c` (`parsePoli`, `liberaPoli`).	`"3xy + 4yz + 2x + 5"` produce 4 monomios y 5 factores.	Pendiente (prototipo probado en la guía).
F4	Secuencial en C: `gradSec` y `evalSec` en `src/secuencial.c`, con medición de tiempo.	Imprime `grad = (8, 15, 8)`; error 0 frente al oráculo.	Parcial: según la conversación del 18-oct existe `gradienteSecuencial.c` con matriz densa fuera del repo [**por verificar**].

Fase	Qué hacer y ficheros	Entregable verificable	Estado
F5	Secuenciales en C++ (`src/secuencial.cpp`) y Rust (`rust/src/main.rs`).	Las tres versiones dan la misma salida que el oráculo.	Pendiente (`secuencial.rs` solo tiene un comentario).
F6	Batería de pruebas: `tests/test.sh` y objetivo `make test`.	`make test` termina con código 0.	Pendiente.
F7	OpenMP en `src/omp.c`: variantes *reduction*, *atomic*, privatización y dos fases; paralelismo por puntos y anidado; `omp_get_wtime`.	Error relativo $\leq 10^{-12}$ frente al secuencial en todas las variantes y tabla de tiempos por hilos.	Pendiente (prototipo probado en la guía).
F8	CUDA base en `src/grad.cu`: un hilo por monomio con `atomicAdd`, desglose H2D, kernel, D2H y total con `cudaEvent`.	Mismo gradiente que el secuencial ($\leq 10^{-12}$) y CSV de tiempos por tamaño de bloque.	Pendiente (prototipo compila para `sm_89`).
F9	CUDA optimizado (opcional): dos fases sin atómicos, `x` en memoria compartida, monomios ordenados por longitud, memoria anclada.	Cada optimización con medida antes y después; se descartan las que no mejoran.	Pendiente.
F10	Sistemas y Jacobiano: `src/jac.c`, `src/jac.cu`, `gen/genSistema.py`.	El sistema del enunciado da las filas $(6, 3, 3)$, $(4, 27, 54)$, $(9, 24, 25)$ en $(1, 2, 3)$.	Pendiente (prototipo secuencial probado).
F11	Benchmarks: `bench/bench.sh`, `bench/graficas.py`, `resultados/*.csv`, `figuras/*.png`.	CSV con mediana y desviación; gráficas de speedup y eficiencia.	Pendiente (prototipo probado).
F12	Contenedor: `Dockerfile` y, si el servidor no permite Docker, `grad.def` para Apptainer.	`docker run --gpus all ...` ejecuta `make test`.	Pendiente.
F13	Registro de IA: `docs/prompts.md` con ámbito, prompts, soluciones y valoración por IA.	Una entrada por interacción relevante, con fecha y versión del código.	Parcial: hay conversaciones previas sin registrar.
F14	Memoria final en `docs/memoria/` con el esquema oficial (Abstract a Bibliografía).	PDF con todas las secciones y resultados del servidor.	Pendiente.
F15	Reproducibilidad: `reproduce.sh` y `resultados/entorno.txt`.	Un único comando regenera CSV y figuras.	Pendiente.

0.1 Diagnóstico del estado actual

Repositorio. La rama `master` tiene 3 commits. El último (`2a655c4`, 8-oct-2026) crea `main.cpp`, `paralelo.cpp`, `CUDA.cpp`, `secuencial.cpp` (vacío) y `secuencial.rs`, todos con solo un comentario, y *borra* `README.md` y `ePract.md`. La rama remota `claude/fervent-bell-1ekfj8` añade `generador.py`, que escribe dos CSV (`monomios.csv` con coeficientes y `factores.csv` con `monomio, variable, exp`)

usando NumPy y Polars.

Problemas detectados en lo existente.

- Las extensiones no encajan con la decisión de usar C: `paralelo.cpp` y `CUDA.cpp` se compilarían como C++, y `nvcc` solo trata como CUDA los ficheros `.cu` (o con `-x cu`).
- `generador.py` usa `eMax = 100` y hasta `cMax = 100` factores por monomio. Con puntos de evaluación fuera de $[0.5, 1.5]$ el producto desborda `double` (por ejemplo $2^{100 \cdot 100}$). Además genera nombres de 5 caracteres en base 62 que pueden empezar por dígito (`"00a1Z"`): en formato texto serían indistinguibles del coeficiente.
- El generador no escribe el punto de evaluación ni un oráculo, por lo que no permite validar.

Erratas e inconsistencias del enunciado (conviene confirmarlas con el profesor):

- Los pesos no coinciden: el índice dice 40 % OpenMP, 40 % CUDA y 20 % documentación; el cuerpo dice 30 % OpenMP, 20 % CUDA, 20 % sistemas, 15 % IA y 15 % documentación (suma 100 %). Esta guía usa los pesos del cuerpo.
- Los objetivos hablan de *sistemas de ecuaciones lineales*, pero el problema es polinómico (no lineal) y no se pide resolver sistemas, solo derivarlos.
- Se citan los apartados 1.1.1 a 1.1.3, que en el documento son 2.1.1 a 2.1.3.
- En el sistema de ejemplo `3x2` y `yz3` significan $3x^2$ e yz^3 (se perdió el superíndice), y la segunda ecuación repite el término $2x$; el parser debe sumarlos.
- La tarea 6 (contenedores) no tiene peso asignado; la convocatoria de recuperación indica junio-julio de 2026, anterior a la fecha actual; varias referencias (teorema de los cuatro colores, isomorfismo de grafos, redes sociales) no tienen relación con la práctica.
- La fecha de entrega aparece como `TAREA DE ENTREGA`. **Supuesto:** entrega final a finales de enero de 2027 y entrega opcional de OpenMP a principios de diciembre de 2026.

Pesos y lo que se evalúa, según el cuerpo del enunciado:

Bloque	Peso	Condición crítica
OpenMP (2.1.1)	30 %	Tiempos con `omp_get_wtime`, escalabilidad y estudio de anidado
CUDA (2.1.2)	20 %	Si los resultados no coinciden con el secuencial no se corrige el resto
Sistemas (2.1.3)	20 %	Adaptar estructuras, regiones OpenMP y kernels
IA y prompts (2.1.4)	15 %	Calidad y contexto de los prompts, evolución incremental
Documentación (2.1.5)	15 %	Esquema fijo y citas

0.2 Hardware

Elemento	Portátil de desarrollo	Servidor de prácticas
CPU	Intel Core i5-13420H: 8 núcleos (4 P y 4 E), 12 hilos **[por verificar]**	Desconocida
GPU	RTX 4050 Laptop, arquitectura Ada Lovelace, capacidad de cómputo 8.9, 6 GB GDDR6 **[por verificar]**	Desconocida
FP64 en GPU	En GPUs GeForce el rendimiento en `double` es una fracción pequeña del de `float` (del orden de 1/64) **[por verificar]**	Depende del modelo: una GPU de centro de datos tiene FP64 mucho mayor
RAM	16 GB	Desconocida
Software	CachyOS, driver propietario de NVIDIA	Versiones de gcc, nvcc y driver desconocidas

Primera tarea en el servidor: guardar la salida de estos comandos en `resultados/entorno.txt`. La sección de resultados de la memoria debe salir de aquí, no de esta guía.

```
lscpu; nproc; free -h; gcc --version; nvcc --version
nvidia-smi --query-gpu=name,compute_cap,memory.total,clocks.max.sm --format=csv
echo | gcc -fopenmp -dM -E - | grep _OPENMP
```

La macro `_OPENMP` devuelve la fecha de la especificación que soporta el compilador (por ejemplo 201511 para OpenMP 4.5). Con `capacidad de cómputo` sabrás qué `-arch=sm_XY` usar.

0.3 Índice de temas

Tema	Contenido	Fases
1. Fundamentos matemáticos (1)	Polinomios, derivadas, gradiente, métodos de derivación, coma flotante	F3, F4, F6, F10
2. Datos, formatos y generadores (2)	Formato texto, parser, CSR, CSC, SoA, generadores, oráculo	F1, F2, F3
3. Algoritmo secuencial (3)	Potencias, prefijo-sufijo, coste, C, C++ y Rust, compilación	F4, F5
4. Rendimiento y medida (4)	Tiempos, speedup, Amdahl, Gustafson, roofline, perfiladores	F7, F8, F9, F11
5. OpenMP (5)	Fork-join, carreras, reducciones, schedule, anidado, afinidad	F7, F10
6. CUDA: fundamentos (6)	Arquitectura, kernels, memoria, transferencias, eventos, atómicos	F8, F10
7. CUDA: optimización (7)	Ocupación, coalescing, shared, shuffles, divergencia, streams	F9
8. Sistemas y Jacobiano (8)	Estructura de datos, paralelización y almacenamiento	F10
9. Validación y benchmarking (9)	Tests, tolerancias, metodología, gráficas	F2, F6, F11, F15
10. Herramientas y entorno (10)	Git, Make, Docker, Apptainer, Python, servidor	F0, F12, F15
11. IA generativa y memoria (11)	Ingeniería de prompts, registro, redacción	F13, F14
12. Estado del arte (12)	Diferenciación automática, evaluación de polinomios en GPU	F14 y todas

1 Fundamentos matemáticos

Tema 1: contenidos que necesitas para las fases F3, F4, F6 y F10.

1.1 Monomio y polinomio multivariable

Qué es. Un monomio en N variables es $c\,x_1^{e_1}x_2^{e_2}\cdots x_N^{e_N}$ con coeficiente $c \in \mathbb{R}$ y exponentes $e_j \in \mathbb{N}$. Un polinomio es una suma de M monomios:

$$f(\mathbf{x}) = \sum_{m=1}^{M} c_m \prod_{j=1}^{N} x_j^{e_{mj}}.$$

Llamamos k_m al número de variables con $e_{mj} > 0$ en el monomio m (sus *factores*) y $F = \sum_m k_m$ al total de factores. Un término constante es un monomio con $k_m = 0$.

Por qué lo usamos. El enunciado pide funciones *arbitrarias compuestas por múltiples monomios y variables*. Los polinomios son exactamente eso y permiten derivar de forma exacta sin cálculo simbólico general. En casos realistas $k_m \ll N$: cada monomio usa pocas variables. Esa dispersión decide la estructura de datos (tema 2) y el coste ($O(F)$ y no $O(MN)$).

Cómo se usa. En el ejemplo $f = 3xy + 4yz + 2x + 5$: $N = 3$, $M = 4$, $k = (2, 2, 1, 0)$ y $F = 5$.

Alternativas. Funciones generales (senos, exponenciales) exigirían un árbol de expresión y diferenciación automática o simbólica. Son más generales, pero más lentas y fuera del enunciado, que fija monomios.

1.2 Derivada parcial y gradiente

Qué es. La derivada parcial $\partial f/\partial x_j$ mide la variación de f al mover solo x_j. El gradiente es el vector de las N derivadas parciales: $\nabla f = (\partial f/\partial x_1, \ldots, \partial f/\partial x_N)$.

Por qué lo usamos. Es lo que hay que calcular. Por linealidad de la derivada, el gradiente es la suma de los gradientes de cada monomio. Eso permite repartir monomios entre hilos y sumar contribuciones.

Cómo se usa. Para un monomio, por las reglas del producto y de la potencia:

$$\frac{\partial}{\partial x_j}\Big(c_m \prod_l x_l^{e_{ml}}\Big) = \begin{cases} c_m\, e_{mj}\, x_j^{e_{mj}-1} \prod_{l\neq j} x_l^{e_{ml}} & \text{si } e_{mj} > 0, \\ 0 & \text{si } e_{mj} = 0. \end{cases}$$

Así, $\partial f/\partial x_j = \sum_{m:\, e_{mj}>0}(\ldots)$. En el ejemplo, $\nabla f = (3y + 2,\ 3x + 4z,\ 4y)$ y $\nabla f(1, 2, 3) = (8, 15, 8)$, que es el caso de prueba obligatorio.

Alternativas. Ninguna para la definición; las alternativas están en cómo calcularla (concepto 1.3).

1.3 Métodos para calcular derivadas

Qué es. Hay cuatro familias:

- **Simbólica**: manipula la expresión y obtiene la fórmula (SymPy, Mathematica).
- **Analítica estructurada**: aplica la fórmula del concepto 1.2 sobre una estructura conocida (la nuestra).
- **Diferencias finitas**: aproxima $\partial f/\partial x_j \approx (f(\mathbf{x} + h\mathbf{e}_j) - f(\mathbf{x} - h\mathbf{e}_j))/2h$.

- **Diferenciación automática (AD)**: aplica la regla de la cadena a cada operación del programa, en modo directo o inverso.

Por qué lo usamos. Usamos la analítica estructurada porque es exacta (salvo redondeo), cuesta $O(F)$ y su patrón de acceso es regular, lo que la hace paralelizable. El enunciado pide además automatizar el procedimiento paso a paso del ejemplo, que es justo derivar término a término.

Cómo se usa. El algoritmo concreto está en el tema 3. El modo inverso de AD aplicado a un producto $\prod_t p_t$ da los productos prefijo y sufijo que usaremos (ejemplo de Speelpenning, en Griewank y Walther, 2008).

Alternativas.

- Diferencias finitas: triviales de programar, pero cuestan $2N$ evaluaciones completas ($O(NF)$) y tienen error de truncamiento $O(h^2)$ más error de redondeo $O(\varepsilon/h)$. El h óptimo es del orden de $\varepsilon^{1/3} \approx 6 \cdot 10^{-6}$ y deja unos 10 dígitos correctos. Sirven como comprobación cruzada, no como base.
- Simbólica: exacta en aritmética racional, pero muy lenta y con crecimiento de expresiones. La usamos solo como oráculo para casos pequeños.
- AD con librerías (JAX, PyTorch, Enzyme): general, pero añade dependencias, no se adapta a la estructura dispersa y no es lo que se evalúa.
- Paso complejo $\operatorname{Im} f(\mathbf{x} + ih\mathbf{e}_j)/h$: exacto a precisión de máquina, pero sigue costando N evaluaciones.

1.4 Aritmética en coma flotante

Qué es. IEEE 754 `double` tiene 53 bits de mantisa: redondeo unitario $u = 2^{-53} \approx 1.11 \cdot 10^{-16}$ y rango aproximado $10^{\pm 308}$. Cada operación introduce un error relativo de como mucho u. La suma *no es asociativa*: $(a + b) + c$ puede diferir de $a + (b + c)$.

Por qué lo usamos. Tiene tres consecuencias directas:

1. Las versiones paralelas suman en otro orden. Su resultado puede diferir del secuencial en los últimos bits, así que la comparación ha de usar una tolerancia, no la igualdad.
2. Los atómicos y la planificación dinámica hacen que el orden cambie entre ejecuciones: el resultado no es determinista.
3. Con exponentes altos se desborda: $1.5^{100} \approx 4 \cdot 10^{17}$ cabe, pero un monomio de 100 factores con $x = 2$ y $e = 100$ vale 2^{104}, que es `inf`.

Cómo se usa. La cota de error de una suma de n términos es aproximadamente $(n-1)\,u \sum |t_i|$. Si hay signos mezclados, $\sum |t_i|$ puede ser mucho mayor que $|\sum t_i|$ (cancelación) y el error relativo crece. Por eso el criterio recomendado es

$$|g_j - g_j^{\text{ref}}| \le \text{rtol} \cdot \max(1, |g_j^{\text{ref}}|),$$

con $\text{rtol} = 10^{-12}$ para `double` con coeficientes positivos y puntos en $[0.5, 1.5]$. Si aparecen cancelaciones, se compara contra $\sum |t_i|$. Para evitar desbordamientos, los generadores deben limitar $e \le 5$, $k \le 16$ y $x \in [0.5, 1.5]$: el peor caso es $1.5^{80} \approx 1.2 \cdot 10^{14}$ y $0.5^{80} \approx 8 \cdot 10^{-25}$, ambos representables.

Trampa habitual

Comparar con `==` entre la versión secuencial y la paralela, o usar `-ffast-math`, que permite reordenar operaciones y rompe la reproducibilidad. Si usas `-ffast-math`, mide el error antes de adoptarlo.

Alternativas. Suma de Kahan o suma por parejas (*pairwise*): reducen el error a casi $O(u)$ a cambio de 2 a 4 veces más operaciones. Es razonable solo si se observan cancelaciones. `long double` o aritmética racional son útiles solo para el oráculo.

2 Datos, formatos y generadores

Tema 2: contenidos que necesitas para las fases F1, F2 y F3.

2.1 Formato de entrada en texto

Qué es. Un formato textual que reproduce la notación del enunciado: `f(x, y, z) = 3xy + 4yz + 2x + 5` y ecuaciones como `3x^2 + yz + z + 5 = 2`.

Por qué lo usamos. El enunciado trabaja así y facilita depurar a mano. Para que sea inequívoco con muchas variables, fijamos esta regla: **una variable es una letra seguida opcionalmente de dígitos** (`x`, `y`, `x12`). Así `xy` es $x \cdot y$ y `x12x3^2` es $x_{12}x_3^2$. Un nombre nunca empieza por dígito, de modo que no se confunde con el coeficiente.

Cómo se usa. Gramática (EBNF) que implementa `poli.c`:

```
ecuacion := expr [ "=" numero ]
expr     := [ "+" | "-" ] termino { ( "+" | "-" ) termino }
termino  := [ numero ] { [ "*" ] variable [ "^" entero ] }
variable := letra { digito }
```

Fichero de función: línea 1 con los nombres de las variables (fija el orden del gradiente), línea 2 con la expresión, línea 3 con el punto. Fichero de sistema: línea 1 con las variables, línea 2 con K y K líneas de ecuación; el punto, al final. Si una variable se repite dentro de un monomio (`xx`), se suman sus exponentes; los monomios repetidos (`2x + ... + 2x`) se dejan separados porque la derivada es lineal.

Alternativas.

- CSV de dos tablas (el `generador.py` de la rama): fácil de generar con Polars, pero más lento de leer en C que un binario y sin la notación del enunciado.
- Binario (cabecera con N, M, F y los arrays volcados con `fwrite`): carga casi a la velocidad del disco. Recomendado para los casos grandes de benchmark, siempre que el texto siga siendo el formato de validación.
- Formato con separador explícito (`3*x0*x1^2`): más robusto, pero se aleja del enunciado. El parser lo acepta igualmente, porque `*` es opcional.

2.2 Parser

Qué es. El código que convierte el texto en la estructura de datos.

Por qué lo usamos. Es la pieza que hace la solución *dinámica*: el número de variables y monomios solo se conoce al leer.

Cómo se usa. Analizador de una pasada con un puntero que avanza (*scanner* manual). Coste $O(L)$ en la longitud del texto, más la búsqueda del nombre. En el prototipo la búsqueda es lineal ($O(N)$ por factor): vale para validar, pero para $N = 10^4$ hay que usar una tabla hash o, mejor, extraer el índice del propio nombre (`x12` es la variable 12). Fragmento central:

```
while (*c == '*' || isalpha((unsigned char)*c)) {
```

```
    if (*c == '*') { c++; continue; }
    const char *s = c++;
    while (isdigit((unsigned char)*c)) c++;
    int v = buscaVar(s, (int)(c - s), vNombres, nVars);
    if (v < 0) return -1;
    int e = 1;
    if (*c == '^') { c++; e = (int)strtol(c, (char **)&c, 10); }
    ...
}
```

Alternativas. `flex/bison` (potente, pero excesivo para esta gramática), expresiones regulares (`regex.h`, lentas y frágiles con signos) o parsear en Python y pasar binario a C (rápido de escribir, pero el enunciado pide una función C que represente la función).

Trampa habitual

Medir el tiempo del parser junto al del gradiente. En casos grandes el parseo de texto puede costar más que el cálculo y ocultaría el speedup. Mide y reporta las dos cosas por separado.

2.3 Representación densa frente a dispersa (CSR)

Qué es.

- **Densa**: matriz $M \times N$ de exponentes (la decisión inicial del 18-oct).
- **COO**: lista de tripletas (monomio, variable, exponente).
- **CSR** (*compressed sparse row*): `vOff[M+1]` indica dónde empiezan los factores de cada monomio en `vIdVar[F]` y `vExp[F]`.
- **ELL**: cada monomio con $k_{\text{máx}}$ huecos.
- **CSC**: la transpuesta, agrupada por variable.

Por qué lo usamos. Usamos CSR porque cuesta $O(M + F)$ en memoria y en recorrido, frente a $O(MN)$ de la densa. Con los tamaños de benchmark propuestos (k uniforme en $[1, 16]$, media 8.5):

Caso	N	M	F	CSR	Extra dos fases	Densa (`int32`)
S	10^3	10^4	$8.5 \cdot 10^4$	0.8 MB	1.0 MB	0.04 GB
M	10^4	10^6	$8.5 \cdot 10^6$	80 MB	102 MB	40 GB
L	10^5	10^7	$8.5 \cdot 10^7$	800 MB	1020 MB	4000 GB

CSR $= 8M + 4(M + 1) + 8F$ bytes; la columna *extra dos fases* suma `vContrib` ($8F$), la CSC ($4F$) y sus desplazamientos ($4(N + 1)$). La densa es inviable desde el caso M, y en una GPU de 6 GB solo caben S y M en forma densa (y M ni siquiera). En CSR, el caso L entero ocupa unos 1.8 GB: cabe en 6 GB.

Cómo se usa. Estructura usada en toda la guía (`poli.h`):

```
typedef struct {
    int nVars;
    int nMon;
    int nFact;
    double *vCoef;
    int *vOff;
    int *vIdVar;
    int *vExp;
```

```
} Poli;
```

El ejemplo $3xy + 4yz + 2x + 5$ queda como `vCoef = {3,4,2,5}`, `vOff = {0,2,4,5,5}`, `vIdVar = {0,1,1,2,0}` y `vExp = {1,1,1,1,1}`.

Alternativas.

- Densa: acceso trivial y buena si $k_m \approx N$ (funciones muy densas y pequeñas, como el ejemplo). Medir la densidad $F/(MN)$: si supera un 30 % aproximadamente, la densa puede competir **[por verificar]**.
- ELL: accesos perfectamente alineados en GPU (coalescing), pero desperdicia $M(k_{\text{máx}} - \bar{k})$ huecos: casi el doble con $k \in [1, 16]$.
- COO: más simple de construir, pero repite el índice de monomio ($4F$ bytes más) y obliga a atómicos.
- Tipos más pequeños (`uint8_t` para exponentes si $e \leq 255$, `uint16_t` para variables si $N \leq 65535$) reducen tráfico de memoria. Es una optimización válida en un problema limitado por memoria (tema 4).

2.4 AoS frente a SoA

Qué es. *Array of Structures* (un array de `struct {int var; int exp;}`) frente a *Structure of Arrays* (`vIdVar[]` y `vExp[]` separados).

Por qué lo usamos. En GPU, hilos consecutivos leen posiciones consecutivas de cada array (coalescing). En CPU, SoA facilita la vectorización SIMD. Usamos SoA.

Cómo se usa. Ya está así en `Poli`.

Alternativas. AoS de 8 bytes (`int2`) también se lee bien en GPU y mejora la localidad en CPU. Es una variante a medir en F9, no una decisión crítica.

2.5 Generadores de datos

Qué es. Programas Python que crean funciones y sistemas aleatorios con parámetros N, M, $k_{\text{máx}}$, $e_{\text{máx}}$, K y semilla.

Por qué lo usamos. Hacen falta casos grandes para medir y casos pequeños con respuesta conocida para validar. El enunciado admite *ficheros de entrada elaborados por el alumno.*

Cómo se usa. Reglas para ambos generadores:

- Generador de números con semilla explícita (`numpy.random.default_rng(semilla)` o `random.Random(semilla)`), nunca el estado global.
- Rangos seguros: $c \in [1, 9]$, $e \in [1, 5]$, $k \in [1, 16]$, punto en $[0.5, 1.5]$ (concepto 1.4).
- Variables `x0` a `x{N-1}`, sin repetir dentro de un monomio (`rng.sample`).
- Escritura en texto (validación) y en binario (benchmarks grandes).
- Generar por bloques, como ya hace `generador.py`, para no agotar memoria con $M = 10^7$.

Alternativas. Generar en C (más rápido, pero duplicas lógica). Usar el `generador.py` existente cambiando `eMax` a 5, `cMax` a 16 y los nombres a `x<id>`, y añadiéndole el punto: es la opción más barata.

2.6 Oráculo con SymPy

Qué es. SymPy es una librería de cálculo simbólico en Python. Deriva la expresión y la evalúa con aritmética racional exacta si el punto es entero o racional.

Por qué lo usamos. Da una referencia independiente de nuestro código. Es la mejor defensa ante el requisito eliminatorio de CUDA (*si los resultados no coinciden no se sigue corrigiendo*).

Cómo se usa. Prototipo probado en `docs/guia/codigo/oraculo.py`. Con puntos enteros en $\{-2, -1, 1, 2, 3\}$ el gradiente de referencia es entero, y la versión C dio error 0 en dos casos ($N = 50$, $M = 200$, semilla 7; $N = 300$, $M = 2000$, semilla 11):

```
vSim = sp.symbols(vNombres)
sust = dict(zip(vSim, vPunto))
vGrad = [sp.diff(expr, s).subs(sust) for s in vSim]
```

Alternativas.

- Para casos grandes SymPy es demasiado lento. Se usa un oráculo de dos niveles: el secuencial en C, validado contra SymPy en casos pequeños, pasa a ser la referencia de las versiones paralelas en los grandes.
- NumPy con `float128` o `mpmath` sirven como oráculo de precisión alta.
- JAX como oráculo de AD: posible, pero añade una dependencia pesada.

3 Algoritmo secuencial

Tema 3: contenidos que necesitas para las fases F4 y F5.

3.1 Cálculo de potencias enteras

Qué es. Calcular x^e con e entero.

Por qué lo usamos. Es la operación más repetida: dos veces por factor. Su coste domina el cálculo.

Cómo se usa. Exponenciación por cuadrados, $O(\log e)$ multiplicaciones (Knuth, TAOCP vol. 2, 4.6.3). Para la derivada se calcula $b = x^{e-1}$ una sola vez y se obtienen $e \cdot b$ y $x^e = b \cdot x$:

```
double powi(double x, int e) {
    double r = 1.0;
    while (e > 0) {
        if (e & 1) r *= x;
        x *= x;
        e >>= 1;
    }
    return r;
}
```

Alternativas.

- `pow()` de `math.h`: general (exponentes reales), pero más lenta y con posibles errores de 1 ulp [por verificar]. En CUDA, `pow` en `double` es especialmente cara.
- Bucle de e multiplicaciones: igual de bueno para $e \leq 3$, peor para exponentes grandes.
- Tabla precomputada `mPot[v][e]` para todas las variables y exponentes ($N \cdot e_{\text{máx}}$ valores): convierte cada potencia en una lectura. Conviene si $N \cdot e_{\text{máx}} \cdot 8$ bytes cabe en caché (por

ejemplo $10^4 \cdot 5 \cdot 8 = 400$ KB) y si cada variable aparece en muchos monomios. Se mide en F9.

3.2 Algoritmo del gradiente: ingenuo, división y prefijo-sufijo

Qué es. Tres formas de obtener, para cada factor t de un monomio, el producto del resto de factores:

- **Ingenuo**: recalcula el producto de los otros $k-1$ factores. $O(k^2)$ por monomio.
- **División**: calcula $P = \prod_t p_t$ y usa P/p_t. $O(k)$, pero falla si algún $x = 0$ y pierde precisión cerca de cero.
- **Prefijo-sufijo**: $\text{pre}_t = \prod_{s<t} p_s$ y $\text{suf}_t = \prod_{s>t} p_s$, en dos pasadas. $O(k)$, sin divisiones y robusto.

Por qué lo usamos. Usamos prefijo-sufijo: es el modo inverso de AD aplicado al producto. La diferencia es medible: en la máquina de pruebas el algoritmo ingenuo con 2 hilos fue 3 veces *más lento* que el prefijo-sufijo con 1 hilo (sección 4).

Cómo se usa. Implementación de referencia (`poli.c`, probada con el ejemplo y con el oráculo):

```
void gradSec(const Poli *p, const double *vX, double *vGrad) {
    double vPre[KMAX], vDer[KMAX];
    memset(vGrad, 0, sizeof(double) * p->nVars);
    for (int m = 0; m < p->nMon; m++) {
        int ini = p->vOff[m], k = p->vOff[m + 1] - ini;
        double acum = p->vCoef[m];
        for (int t = 0; t < k; t++) {
            double x = vX[p->vIdVar[ini + t]];
            int e = p->vExp[ini + t];
            double base = powi(x, e - 1);
            vPre[t] = acum;
            vDer[t] = e * base;
            acum *= base * x;
        }
        double suf = 1.0;
        for (int t = k - 1; t >= 0; t--) {
            int v = p->vIdVar[ini + t];
            double x = vX[v];
            int e = p->vExp[ini + t];
            vGrad[v] += vPre[t] * vDer[t] * suf;
            suf *= powi(x, e);
        }
    }
}
```

Salida del programa `ejemplo.c`: `f = 37` y `grad = (8, 15, 8)`.

Trampa habitual

`KMAX` limita los factores por monomio (64 en el prototipo). Si el generador produce $k >$`KMAX` (el `generador.py` actual llega a 100), el programa escribe fuera del array. Valida $k_{\text{máx}}$ al leer o reserva el buffer dinámicamente.

Alternativas.

- La segunda pasada recalcula `powi(x, e)`. Guardar también p_t en un tercer buffer ahorra esa potencia a cambio de memoria local. En GPU puede no compensar por la presión de registros: se mide.
- Horner multivariable reduce multiplicaciones al evaluar f, pero requiere reordenar los monomios en un árbol. Es más complejo y paraleliza peor.

3.3 Coste y memoria del secuencial

Qué es. Tiempo $O(F \log e_{\text{máx}})$ y memoria $O(M + F + N)$.

Por qué lo usamos. Permite predecir tiempos y elegir tamaños: el tiempo debe crecer linealmente con F. Si no lo hace, hay un problema de caché (por ejemplo, N demasiado grande para que `vX` quepa en L2).

Cómo se usa. Por factor: 2 llamadas a `powi` (unas 3 a 6 multiplicaciones cada una con $e \leq 5$), 4 multiplicaciones y 1 suma; se leen 8 bytes (`vIdVar` y `vExp`) más un acceso irregular a `vX` y otro a `vGrad`. En la máquina de pruebas, $F = 8.5 \cdot 10^5$ tardó unos 14 ms (unos 16 ns por factor).

Alternativas. No hay algoritmo asintóticamente mejor: cualquier método debe leer los F factores.

3.4 C, C++ y Rust

Qué es. Los tres lenguajes de las versiones secuenciales.

Por qué lo usamos. C es la base: OpenMP y CUDA se integran sin intermediarios y das control total de la memoria. C++ aporta `std::vector` y RAII. Rust aporta seguridad de memoria. Las tres versiones sirven como comprobación cruzada y como comparación de rendimiento.

Cómo se usa. Mantén la misma estructura CSR y el mismo algoritmo en los tres para que la comparación sea justa. Versión Rust probada (`grad.rs`, salida `[8.0, 15.0, 8.0]`):

```
fn gradSec(p: &Poli, vX: &[f64]) -> Vec<f64> {
    let mut vGrad = vec![0.0; p.nVars];
    let mut vPre = Vec::new();
    for m in 0..p.vCoef.len() {
        let (ini, fin) = (p.vOff[m], p.vOff[m + 1]);
        vPre.clear();
        let mut acum = p.vCoef[m];
        for t in ini..fin {
            let x = vX[p.vIdVar[t]];
            let base = x.powi(p.vExp[t] - 1);
            vPre.push(acum * p.vExp[t] as f64 * base);
            acum *= base * x;
        }
        let mut suf = 1.0;
        for t in (ini..fin).rev() {
            vGrad[p.vIdVar[t]] += vPre[t - ini] * suf;
            suf *= vX[p.vIdVar[t]].powi(p.vExp[t]);
        }
    }
    vGrad
}
```

Rust exige `#![allow(non_snake_case)]` para respetar tu nomenclatura camelCase, porque su convención es `snake_case`.

Alternativas.

- En Rust, la paralelización natural es `rayon` (`par_iter`); no existe OpenMP nativo, y CUDA desde Rust depende de crates poco maduros **[por verificar]**. Por eso la paralela se queda en C.
- `nvcc` compila el código CUDA como C++. Tu "CUDA en C" será C++ con estilo C: `malloc`, punteros y sin clases. Es normal y aceptado.
- Las comprobaciones de límites de Rust pueden costar algo de tiempo; los iteradores suelen

eliminarlas. Mídelo y explícalo en la memoria en lugar de suponerlo.

3.5 Opciones de compilación

Qué es. Las opciones que controlan optimización, avisos y diagnóstico.

Por qué lo usamos. Las comparaciones de rendimiento solo son válidas con las mismas opciones en todas las versiones.

Cómo se usa. `-O3 -Wall -Wextra -std=c11` para medir, `-g -fsanitize=address,undefined` para los tests y `-fopenmp` para OpenMP. En CUDA: `nvcc -O3 -arch=sm_XY -Xptxas=-v`. Todo el código de la guía compila sin avisos con `-Wall -Wextra`.

Alternativas. `-march=native` activa AVX2 o AVX-512 y suele acelerar, pero el binario solo vale para la máquina donde compilas: compila en el servidor. `-ffast-math`: ver la trampa del concepto 1.4. Clang en lugar de gcc: válido para comparar.

4 Rendimiento y medida

Tema 4: contenidos que necesitas para las fases F7, F8, F9 y F11.

4.1 Medición de tiempos

Qué es. Tiempo de pared (*wall-clock*, lo que tarda en terminar) frente a tiempo de CPU (suma de lo que consumen todos los hilos).

Por qué lo usamos. El speedup se calcula con tiempo de pared. El enunciado aconseja `omp_get_wtime()` en CPU y pide desglosar transferencias y cómputo en GPU.

Cómo se usa. `omp_get_wtime()` en OpenMP, `clock_gettime(CLOCK_MONOTONIC)` en el secuencial de C, `std::chrono::steady_clock` en C++, `std::time::Instant` en Rust y `cudaEventRecord`/`cudaEventElapsedTime` en CUDA (tema 6). Mide solo la región de interés y reporta aparte lectura, parseo y cálculo.

Trampa habitual

`clock()` devuelve tiempo de CPU acumulado de todos los hilos. Con 8 hilos marca unas 8 veces el tiempo real y *oculta* el speedup. Tampoco midas un kernel CUDA con un reloj de CPU sin `cudaDeviceSynchronize()`: el lanzamiento es asíncrono.

Alternativas. `gettimeofday` (obsoleto y no monótono), `rdtsc` (ciclos, poco portable) o `perf stat` (útil para contadores, no para tiempos por región).

4.2 Speedup, eficiencia y escalabilidad

Qué es. Speedup $S_p = T_1/T_p$; eficiencia $E_p = S_p/p$. *Escalado fuerte*: problema fijo y más hilos. *Escalado débil*: problema proporcional a los hilos; se mide si T_p se mantiene constante.

Por qué lo usamos. El enunciado pide un *análisis de escalabilidad* con *análisis crítico*: no basta la tabla, hay que explicar por qué la eficiencia cae.

Cómo se usa. T_1 debe ser el **mejor secuencial**, no la versión paralela ejecutada con 1 hilo. Medida orientativa ($N = 10^3$, $M = 10^5$, $F = 853\,478$, máquina de nube de 2 vCPU, gcc `-O2`, mediana de 3 repeticiones):

Variante	1 hilo (s)	2 hilos (s)	Speedup 2 hilos	Eficiencia
Secuencial prefijo-sufijo	0.0138	–	1.00	–
reduction (ingenuo $O(k^2)$)	0.0801	0.0402	0.34	0.17
atomic (ingenuo $O(k^2)$)	0.0806	0.0456	0.30	0.15
Dos fases (prefijo-sufijo)	0.0203	0.0109	1.27	0.63

Conclusiones que se repetirán en el servidor: un algoritmo peor no se arregla con más hilos, y la versión de dos fases con 1 hilo es más lenta que el secuencial porque hace una pasada de memoria extra (`vContrib`). Su beneficio aparece al crecer los hilos.

Alternativas. Reportar *throughput* (factores por segundo) en lugar de tiempo. Es útil para comparar tamaños distintos y lo recomiendo como columna adicional.

4.3 Leyes de Amdahl y Gustafson

Qué es. Amdahl: si una fracción s es secuencial, $S_p \leq 1/(s + (1-s)/p)$, con techo $1/s$. Gustafson: si el problema crece con p, el speedup escalado es $S_p = s + (1-s)\,p$.

Por qué lo usamos. Explican los resultados. Si el parseo (secuencial) es el 20 % del tiempo total, el speedup global nunca pasa de 5, aunque el cálculo escale perfectamente.

Cómo se usa. Mide la fracción secuencial real (lectura, transposición CSC, reserva de memoria) y superpón la curva de Amdahl en la gráfica de speedup.

Alternativas. Ley de Karp-Flatt (fracción serie experimental $e = (1/S_p - 1/p)/(1 - 1/p)$): si e crece con p, el problema es la sobrecarga paralela y no la parte secuencial. Es muy útil para el análisis crítico.

4.4 Jerarquía de memoria, intensidad aritmética y roofline

Qué es. La intensidad aritmética I es el número de operaciones en coma flotante por byte movido desde memoria. El modelo roofline (Williams et al., 2009) acota el rendimiento: $P \leq \text{mín}(P_{\text{pico}},\, I \cdot B)$, con B el ancho de banda.

Por qué lo usamos. Nuestro problema tiene I baja: por factor se mueven unos 16 a 24 bytes (índice, exponente, lectura de `x` y, en dos fases, escritura de `vContrib`) para unas 10 operaciones, es decir, $I \approx 0.4$ a 0.6 flop/byte. Está **limitado por memoria** en CPU, y probablemente en GPU con `float`. Con `double` en una GeForce, el FP64 reducido puede acercar el límite al cómputo **[por verificar]**. Este diagnóstico decide las optimizaciones: reducir bytes (tipos más pequeños, evitar la segunda pasada) antes que reducir operaciones.

Cómo se usa. Mide el ancho de banda con STREAM (*triad*) en el servidor y compara con tus bytes por segundo efectivos (F·bytes por factor/T). Si estás cerca del 60 a 80 % de STREAM, ya no hay mucho margen.

Alternativas. Contar ciclos o instrucciones con `perf stat` (IPC, fallos de caché) da una visión complementaria a nivel de CPU.

4.5 Cuellos de botella de este problema

- **Accesos irregulares a `vX`**: `vIdVar` es aleatorio. Con $N = 10^4$, `vX` ocupa 80 KB y cabe en L2; con $N = 10^5$ ocupa 800 KB y puede no caber. Mide el tiempo por factor variando N con F fijo.

- **Conflictos de escritura en `vGrad`**: varios monomios contribuyen a la misma variable. Con N pequeño la contención de atómicos se dispara.
- **Desequilibrio de carga**: k_m varía de 1 a 16, así que `schedule(static)` puede repartir mal.
- **Núcleos heterogéneos**: el i5-13420H mezcla núcleos P y E **[por verificar]**. Con reparto estático, los hilos en núcleos E terminan después y frenan a todos.
- ***False sharing***: arrays privados por hilo contiguos que comparten línea de caché (64 bytes).
- **PCIe**: subir 80 MB a la GPU a unos 10 a 25 GB/s cuesta de 3 a 8 ms **[por verificar]**, probablemente más que el kernel. Por eso hay que desglosar los tiempos.

4.6 Perfiladores

Qué es. Herramientas que miden dónde se va el tiempo.

Por qué lo usamos. Para justificar optimizaciones con datos, como pide la parte opcional de CUDA (*análisis profundo de la ejecución*).

Cómo se usa. CPU: `perf stat -e cycles,instructions,cache-misses ./omp` y `perf record`/`perf report`. GPU: `nsys profile ./grad` para la línea temporal (transferencias frente a kernels) y `ncu --set full ./grad` para métricas por kernel (ocupación lograda, ancho de banda, divergencia).

Alternativas. `gprof` (antiguo, instrumenta), Valgrind `cachegrind` (muy lento, simula cachés) e Intel VTune.

Trampa habitual

`nvprof` y Visual Profiler son obsoletos y no admiten GPUs de capacidad 8.0 o superior. Usa Nsight Systems y Nsight Compute.

5 OpenMP

Tema 5: contenidos que necesitas para las fases F7 y F10.

5.1 Modelo fork-join y bucles paralelos

Qué es. OpenMP es una API de directivas (`#pragma omp`) para paralelismo en memoria compartida. Un hilo maestro abre una región paralela (*fork*), un equipo de hilos ejecuta trabajo y se sincroniza al final (*join*).

Por qué lo usamos. El enunciado lo exige (30 %), y el bucle sobre monomios es paralelo de forma natural.

Cómo se usa. `#pragma omp parallel for` reparte las iteraciones. Variables: `shared` por defecto, privadas las declaradas dentro del bucle. Usa `default(none)` mientras desarrollas: obliga a declarar todo y destapa errores. Número de hilos con `OMP_NUM_THREADS` o `omp_set_num_threads`. Compila con `-fopenmp`.

Alternativas. Pthreads (control total, mucho más código), C11 `threads.h`, TBB o `std::thread` o `std::execution::par` en C++. MPI no aplica: el enunciado pide memoria compartida.

5.2 Condiciones de carrera al acumular el gradiente

Qué es. Dos hilos que hacen `vGrad[v] += d` sobre la misma `v` a la vez pueden perder una de las sumas.

Por qué lo usamos. Es el problema central de paralelizar por monomios: es un patrón de *scatter* (dispersión de escrituras).

Cómo se usa. Cinco soluciones, de peor a mejor en general:

1. `#pragma omp critical`: serializa la suma. Correcto, pero escala mal. Úsalo solo como versión de referencia.
2. `#pragma omp atomic`: suma atómica en hardware. Barata si hay pocos conflictos; mala con N pequeño.
3. `reduction(+ : vGrad[:n])` (OpenMP 4.5 o posterior): cada hilo tiene una copia privada de n elementos y se suman al final. Coste extra $O(p \cdot N)$ en memoria y en la fusión: excelente con N pequeño y malo con $N = 10^5$ y muchos hilos.
4. Privatización manual: igual que la anterior, pero controlas la fusión (en paralelo por variables) y el alineamiento (para evitar *false sharing*).
5. **Dos fases sin atómicos**: la fase 1 recorre los monomios y escribe cada contribución en su posición única `vContrib[t]` (no hay conflictos); la fase 2 recorre las variables y suma sus contribuciones con la transpuesta CSC (*gather*). Es determinista y escala bien, a cambio de $12F$ bytes extra.

Variante de dos fases (prototipo probado, error relativo $3.8 \cdot 10^{-16}$):

```
#pragma omp parallel
{
    double vPre[64];
    #pragma omp for schedule(dynamic, 256)
    for (int m = 0; m < p->nMon; m++) {
        int ini = p->vOff[m], k = p->vOff[m + 1] - ini;
        double acum = p->vCoef[m];
```

```
        for (int t = 0; t < k; t++) {
            double base = powi(vX[p->vIdVar[ini + t]], p->vExp[ini + t] - 1);
            vPre[t] = acum * p->vExp[ini + t] * base;
            acum *= base * vX[p->vIdVar[ini + t]];
        }
        double suf = 1.0;
        for (int t = k - 1; t >= 0; t--) {
            vContrib[ini + t] = vPre[t] * suf;
            suf *= powi(vX[p->vIdVar[ini + t]], p->vExp[ini + t]);
        }
    }
    #pragma omp for schedule(static)
    for (int v = 0; v < p->nVars; v++) {
        double s = 0.0;
        for (int j = vOffVar[v]; j < vOffVar[v + 1]; j++) s += vContrib[vIdFact[j]];
        vGrad[v] = s;
    }
}
```

La barrera implícita al final del primer `omp for` garantiza que `vContrib` está completo antes de la fase 2. La transpuesta (`transpone` en `omp.c`) se construye una vez con un recuento y una suma prefija, coste $O(F + N)$. Si evalúas muchos puntos con la misma función, se amortiza.

Alternativas. Paralelizar *por variables* directamente (cada hilo calcula una $\partial f/\partial x_j$ recorriendo la CSC) evita atómicos sin `vContrib`, pero recalcula el producto del monomio una vez por cada variable: $O(\sum k_m^2)$. Es la variante natural para el estudio de anidado "variables frente a monomios" que pide el enunciado.

Trampa habitual

La reducción sobre arrays con `vGrad[:n]` necesita que `n` sea una variable local, no `p->nVars`, y crea en la pila de cada hilo una copia de n doubles: con $N = 10^6$ son 8 MB por hilo y pueden desbordar la pila (ajústala con `OMP_STACKSIZE`).

5.3 Planificación (`schedule`)

Qué es. Cómo se reparten las iteraciones: `static` (bloques fijos, sin sobrecarga), `dynamic,c` (cada hilo pide trozos de c iteraciones), `guided` (trozos decrecientes) o `auto`/`runtime`.

Por qué lo usamos. El coste de cada monomio es proporcional a k_m, que varía, y los núcleos P y E tienen velocidades distintas. Es un caso típico de desequilibrio.

Cómo se usa. Usa `schedule(runtime)` en el código y cambia la política con `OMP_SCHEDULE="dynamic,256"` desde el script de benchmark. Así comparas sin recompilar. Barre $c \in \{16, 64, 256, 1024, 4096\}$.

Alternativas. Equilibrar a priori: repartir por número de factores en lugar de por monomios (cada hilo recibe un rango de `vOff` con unos F/p factores). Es estático y sin sobrecarga, y suele igualar a `dynamic`. Se busca con búsqueda binaria en `vOff`.

5.4 Paralelismo anidado

Qué es. Regiones paralelas dentro de regiones paralelas: por ejemplo, un nivel externo sobre puntos de evaluación (o ecuaciones) y uno interno sobre monomios.

Por qué lo usamos. El enunciado pide un *estudio de paralelismo anidado* entre variables, monomios y puntos.

Cómo se usa.

```
omp_set_max_active_levels(2);
#pragma omp parallel for num_threads(hExt)
for (int i = 0; i < nPts; i++) {
    #pragma omp parallel for num_threads(hInt) schedule(dynamic, 256)
    for (int m = 0; m < p->nMon; m++) {
    }
}
```

Fragmento orientativo: el cuerpo interno es el de cualquier variante anterior. Diseño del experimento: con T hilos totales, prueba todas las combinaciones $h_{\text{ext}} \times h_{\text{int}} = T$ (por ejemplo 1×12, 2×6, 3×4, 4×3, 6×2, 12×1) con $P \in \{1, 16, 256\}$ puntos. Resultado esperado: con muchos puntos gana el nivel externo (sin conflictos y mejor localidad); con un punto, solo sirve el interno.

Alternativas.

- `collapse(2)`: aplana los dos bucles en uno. Más simple y casi siempre igual o mejor que el anidado real, pero solo si los bucles son rectangulares (puntos × monomios lo es).
- Tareas (`omp taskloop`): reparto dinámico entre niveles. Útil si las ecuaciones de un sistema tienen tamaños muy distintos.

Trampa habitual

`omp_set_nested()` y `OMP_NESTED` están obsoletos desde OpenMP 5.0. Usa `omp_set_max_active_levels` o `OMP_MAX_ACTIVE_LEVELS`. Sin activarlo, la región interna se ejecuta con 1 hilo sin avisar.

5.5 Afinidad de hilos

Qué es. Fijar cada hilo a un núcleo para que no migre y conserve su caché.

Por qué lo usamos. Sin afinidad, las medidas tienen mucha varianza y el sistema operativo puede mover hilos a núcleos E o a hilos SMT del mismo núcleo.

Cómo se usa. `OMP_PROC_BIND=close OMP_PLACES=cores` (ya en `bench.sh`). Comprueba la colocación con `OMP_DISPLAY_AFFINITY=true`. En anidado: `OMP_PROC_BIND=spread,close`.

Alternativas. `taskset` o `numactl` desde fuera. `spread` reparte entre zócalos o NUMA y puede ser mejor en un servidor de dos procesadores.

5.6 Vectorización SIMD

Qué es. Instrucciones que operan sobre 4 u 8 doubles a la vez (AVX2 o AVX-512).

Por qué lo usamos. Es la otra mitad del rendimiento en CPU. Aquí los accesos indirectos (`vX[vIdVar[t]]`) dificultan la vectorización automática.

Cómo se usa. Prueba `#pragma omp simd` en la fase 2 (suma) y comprueba con `-fopt-info-vec`. Los *gathers* de AVX2 existen, pero rara vez ganan mucho.

Alternativas. Reorganizar a ELL por bloques de 8 monomios para vectorizar la fase 1. Es complejo; solo merece la pena si la memoria escrita lo justifica.

6 CUDA: fundamentos

Tema 6: contenidos que necesitas para las fases F8 y F10.

6.1 Arquitectura de una GPU NVIDIA

Qué es. Una GPU tiene decenas de *Streaming Multiprocessors* (SM). Cada SM ejecuta grupos de 32 hilos (*warps*) en modo SIMT: los 32 hilos ejecutan la misma instrucción. Jerarquía de memoria: registros (por hilo), memoria compartida y L1 (por SM, decenas de KB), L2 (compartida, unos MB), memoria global (GB, alto ancho de banda y alta latencia) y memoria constante (64 KB, con caché y eficiente solo si todo el warp lee la misma dirección).

Por qué lo usamos. El rendimiento depende de tener muchos warps listos para ocultar la latencia de memoria (ocupación) y de que los accesos de un warp sean contiguos (coalescing).

Cómo se usa. Consulta las características reales con `cudaGetDeviceProperties` o con el ejemplo `deviceQuery`. Capacidad de cómputo: 8.9 en la RTX 4050 **[por verificar]**; la del servidor está por medir.

Alternativas. OpenCL, SYCL o HIP (AMD) son modelos equivalentes; OpenMP `target` y OpenACC son directivas para GPU. El enunciado exige CUDA.

6.2 Kernels, grid y bloques

Qué es. Un kernel (`__global__`) es una función que ejecutan muchos hilos. Se lanza con `kernel<<<nBloques, tamBloque>>>(...)`. Cada hilo calcula su índice global con `blockIdx.x * blockDim.x + threadIdx.x`.

Por qué lo usamos. Es el mecanismo para asignar trabajo: un hilo por monomio (o por factor, variable o punto).

Cómo se usa. nBloques $= \lceil M/\text{tamBloque} \rceil$, y el hilo comprueba `if (m >= nMon) return;`. Kernel base con un hilo por monomio y `atomicAdd` (`grad.cu`, compila para `sm_89`):

```
__global__ void kMonoAtom(int nMon, const double *__restrict__ vCoef, const int *__restrict__
     vOff,
                          const int *__restrict__ vIdVar, const int *__restrict__ vExp,
                          const double *__restrict__ vX, double *vGrad) {
    int m = blockIdx.x * blockDim.x + threadIdx.x;
    if (m >= nMon) return;
    int ini = vOff[m], k = vOff[m + 1] - ini;
    double vPre[KMAX];
    double acum = vCoef[m];
    for (int t = 0; t < k; t++) {
        double x = __ldg(&vX[vIdVar[ini + t]]);
        int e = vExp[ini + t];
        double base = powiDev(x, e - 1);
        vPre[t] = acum * e * base;
        acum *= base * x;
    }
    double suf = 1.0;
    for (int t = k - 1; t >= 0; t--) {
        int v = vIdVar[ini + t];
        atomicAdd(&vGrad[v], vPre[t] * suf);
        suf *= powiDev(__ldg(&vX[v]), vExp[ini + t]);
    }
}
```

Alternativas. Un hilo por factor (más paralelismo, pero necesita el producto del monomio: dos kernels o reducción segmentada), un warp por monomio (bueno si k es grande, desperdicia hilos si $k \leq 16$) o un hilo por punto (ideal con muchos puntos).

6.3 Gestión de memoria y transferencias

Qué es. La GPU tiene su propia memoria. Hay que reservar (`cudaMalloc`), copiar (`cudaMemcpy`) en ambos sentidos y liberar (`cudaFree`).

Por qué lo usamos. El enunciado exige el desglose transferencia/cómputo/total. En este problema la transferencia de la estructura ($8M + 4M + 8F$ bytes) domina a una sola evaluación del gradiente, que solo devuelve $8N$ bytes.

Cómo se usa. Copia la estructura CSR una vez. Si evalúas varios puntos, copia solo `vX` en cada uno y deja la estructura en la GPU: así amortizas la transferencia y lo puedes argumentar en la conclusión. Reserva memoria anclada (`cudaMallocHost`) para los buffers del host y las copias irán más rápido **[por verificar]**; mide la diferencia.

Alternativas. Memoria unificada (`cudaMallocManaged`): más cómoda, pero las migraciones por fallo de página enturbian el desglose de tiempos que pide el enunciado. Evítala para medir.

6.4 Medida con eventos CUDA

Qué es. `cudaEvent_t` marca instantes en la cola de la GPU; `cudaEventElapsedTime` da los milisegundos entre dos marcas con resolución de unos 0.5 µs **[por verificar]**.

Por qué lo usamos. Es la forma correcta de medir operaciones asíncronas de GPU.

Cómo se usa. Cuatro eventos: antes de H2D, después de H2D, después del kernel y después de D2H (código completo en `grad.cu`):

```
CHECK(cudaEventRecord(ev1));
kMonoAtom<<<nBloques, tamBloque>>>(nMon, dCoef, dOff, dIdVar, dExp, dX, dGrad);
CHECK(cudaGetLastError());
CHECK(cudaEventRecord(ev2));
CHECK(cudaMemcpy(vGrad.data(), dGrad, sizeof(double) * nVars, cudaMemcpyDeviceToHost));
CHECK(cudaEventRecord(ev3));
CHECK(cudaEventSynchronize(ev3));
CHECK(cudaEventElapsedTime(&tKer, ev1, ev2));
```

Trampa habitual

La primera llamada a CUDA crea el contexto y puede tardar cientos de milisegundos. Haz una llamada de calentamiento (por ejemplo `cudaFree(0)`) y una ejecución del kernel que no midas. Comprueba *siempre* los errores: un lanzamiento fallido no avisa hasta que preguntas con `cudaGetLastError`.

Alternativas. Nsight Systems da la misma información de forma gráfica. Úsalo para validar los eventos, no para sustituirlos, porque el enunciado pide números.

6.5 Operaciones atómicas en GPU

Qué es. `atomicAdd(double*, double)` suma de forma indivisible en memoria global o compartida. Existe en hardware para `double` desde la capacidad 6.0.

Por qué lo usamos. Es la forma más directa de resolver la carrera del concepto 5.2 en GPU.

Cómo se usa. Como en `kMonoAtom`. Su coste crece con los conflictos: si $N = 10$ y $M = 10^6$, millones de hilos atacan 10 direcciones. Mide con $N \in \{10, 10^3, 10^5\}$ y F fijo para mostrarlo (experimento E8).

Alternativas. Dos fases sin atómicos (tema 7) y reducción en memoria compartida por bloque antes del atómico global (solo ayuda si los monomios de un bloque comparten variables).

Trampa habitual

La emulación de `atomicAdd` para `double` con `atomicCAS` que aparece en apuntes antiguos solo es necesaria para capacidades menores que 6.0. En GPUs actuales es obsoleta y más lenta.

7 CUDA: optimización

Tema 7: contenidos que necesitas para las fases F9 (parte opcional del 2.1.2).

7.1 Ocupación y recursos por hilo

Qué es. La ocupación es la proporción de warps activos en un SM respecto al máximo. La limitan los registros por hilo, la memoria compartida por bloque y el tamaño de bloque.

Por qué lo usamos. El enunciado propone examinar la salida de `-Xptxas=-v`. Salida real de los kernels de la guía con nvcc 12.0 para `sm_89`:

Kernel	Registros	Pila (bytes)	Observación
`kMonoAtom`	20	256	`vPre[32]` va a memoria local
`kContrib`	23	0	escribe en global, sin array local
`kSumaVar`	40	0	reducción con *shuffles*

Los 256 bytes de pila de `kMonoAtom` son `vPre[KMAX]` con `KMAX` $= 32$ doubles: un array indexado con una variable no cabe en registros y se aloja en memoria local (que físicamente es global, con caché L1). Es tráfico de memoria oculto, y la versión de dos fases lo elimina.

Cómo se usa. `nvcc -O3 -arch=sm_XY -Xptxas=-v grad.cu`. Para calcular la ocupación teórica: `cudaOccupancyMaxActiveBlocksPerMultiprocessor` o `cudaOccupancyMaxPotentialBlockSize` en el propio programa, o la calculadora de ocupación de Nsight Compute. La ocupación *lograda* se mide con `ncu`.

Trampa habitual

La hoja `CUDA_Occupancy_calculator.xls` que cita el enunciado está descontinuada en las versiones recientes del toolkit [por verificar]. Usa la API de ocupación o Nsight Compute y explícalo en la memoria.

Alternativas. `__launch_bounds__(maxHilos, minBloques)` o `-maxrregcount` fuerzan menos registros, pero pueden provocar *spills*. Más ocupación no siempre es más rendimiento si el kernel está limitado por ancho de banda.

7.2 Tamaño de bloque

Qué es. El número de hilos por bloque: un múltiplo de 32 hasta 1024.

Por qué lo usamos. El enunciado pide variarlo explícitamente.

Cómo se usa. Barre $\{32, 64, 128, 256, 512, 1024\}$ pasándolo como argumento (`grad.cu` lo acepta como tercer parámetro). Esperado: 32 rinde mal (pocos warps por SM por el límite de bloques por SM), y la mejora se aplana entre 128 y 512. Interpreta los resultados con la ocupación teórica de cada tamaño.

Alternativas. Elegirlo automáticamente con `cudaOccupancyMaxPotentialBlockSize`. Puedes usarlo como valor por defecto y comparar con el barrido.

7.3 Coalescing y caché de solo lectura

Qué es. Un acceso es coalescente cuando los 32 hilos de un warp leen direcciones contiguas, y se sirve en pocas transacciones de 32 bytes. `__ldg` y `const __restrict__` dirigen las lecturas por la caché de solo lectura o de texturas.

Por qué lo usamos. Con un hilo por monomio, el hilo m lee `vIdVar[vOff[m]+t]`: los hilos de un warp leen posiciones separadas k_m elementos, así que el acceso *no* es coalescente. `vX[vIdVar[...]]` es aleatorio por naturaleza.

Cómo se usa. Opciones, de menor a mayor esfuerzo:

1. `__ldg` en `vX` (ya aplicado).
2. Formato ELL transpuesto: el factor t del monomio m en la posición `t * nMon + m`. Los hilos consecutivos leen posiciones consecutivas y el acceso es perfectamente coalescente, a cambio del relleno.
3. Ordenar los monomios por k_m antes de copiarlos: los warps tienen trabajo homogéneo (menos divergencia) y el relleno ELL se reduce por tramos (*sliced ELL*, al estilo de SpMV en GPU; Bell y Garland, 2009).

Alternativas. Memoria de texturas con objetos de textura: obsoleta como optimización explícita desde que existe `__ldg`.

Trampa habitual

Las *texture references* se eliminaron en CUDA 12. Si un tutorial las usa, está desactualizado; los objetos de textura (`cudaTextureObject_t`) siguen existiendo.

7.4 Memoria compartida (tiling)

Qué es. Memoria rápida por bloque (`__shared__`), gestionada por el programador; hasta 48 KB por bloque por defecto, más configurando el kernel.

Por qué lo usamos. El enunciado sugiere *tiling*. Aquí lo aprovechable es cargar `vX` entero en memoria compartida si cabe: $8N \leq 48$ KB, es decir, $N \leq 6144$. Cada lectura aleatoria de `x` pasa a ser de baja latencia.

Cómo se usa.

```
extern __shared__ double sX[];
for (int i = threadIdx.x; i < nVars; i += blockDim.x) sX[i] = vX[i];
__syncthreads();
```

Fragmento orientativo; el lanzamiento debe pasar `nVars * sizeof(double)` como tercer parámetro de `<<<...>>>`. Para N grande: *tiling* por rangos de variables (procesar solo los factores cuya variable cae en el tramo cargado) exige reordenar los factores por variable y complica mucho el código.

Valóralo solo si `ncu` muestra que `vX` es el cuello de botella.

Alternativas. Memoria constante para `vX`: **no conviene**, porque los hilos de un warp leen variables distintas y la memoria constante serializa los accesos a direcciones diferentes. Sí conviene para escalares comunes. La L1 y la caché de solo lectura ya cachean `vX` sin código extra: mide si la compartida mejora de verdad.

7.5 Dos fases y reducción con shuffles

Qué es. La versión GPU del concepto 5.2: `kContrib` (un hilo por monomio escribe `vContrib`) y `kSumaVar` (un warp por variable suma su lista CSC con `__shfl_down_sync`).

Por qué lo usamos. Elimina atómicos y memoria local, y es determinista. La suma por warp tiene accesos parcialmente coalescentes, porque los 32 hilos leen `vIdFact` consecutivo.

Cómo se usa.

```
__global__ void kSumaVar(int nVars, const int *__restrict__ vOffVar, const int *__restrict__
    vIdFact,
                         const double *__restrict__ vContrib, double *vGrad) {
    int warp = (blockIdx.x * blockDim.x + threadIdx.x) / 32;
    int lane = threadIdx.x % 32;
    if (warp >= nVars) return;
    double s = 0.0;
    for (int j = vOffVar[warp] + lane; j < vOffVar[warp + 1]; j += 32) s += vContrib[vIdFact[
    j]];
    for (int d = 16; d > 0; d >>= 1) s += __shfl_down_sync(0xffffffff, s, d);
    if (lane == 0) vGrad[warp] = s;
}
```

Si cada variable aparece de media en F/N factores, con $F/N < 32$ muchos hilos del warp quedan ociosos: usa un hilo por variable en ese caso. La CSC se puede construir en la GPU con `thrust::sort_by_key` o en el host una sola vez.

Alternativas. Reordenar `vContrib` directamente en orden CSC (escribir en `vContrib[posCsc[t]]`) hace la fase 2 totalmente coalescente, a cambio de escrituras dispersas en la fase 1. Hay que medirlo.

Trampa habitual

`__shfl_down` sin el sufijo `_sync` está obsoleto desde CUDA 9 y no es correcto en arquitecturas Volta o posteriores. Tampoco uses `cudaThreadSynchronize` (obsoleto): usa `cudaDeviceSynchronize`.

7.6 Divergencia de warps

Qué es. Si los hilos de un warp siguen caminos distintos (bucles de distinta longitud), el warp ejecuta todos los caminos y los hilos que no toca esperan.

Por qué lo usamos. El bucle sobre k_m diverge: el warp tarda lo que su monomio más largo. Con k uniforme en $[1, 16]$, la eficiencia media del warp es de un 54 % ($\bar{k}/E[\text{máx}_{32}\, k] \approx 8.5/15.86$).

Cómo se usa. Ordenar monomios por k_m (concepto 7.3) o agrupar en tramos de igual longitud. Mide la métrica *branch efficiency* o *warp execution efficiency* con `ncu`.

Alternativas. Un hilo por factor más una reducción segmentada (todos los hilos con el mismo trabajo): elimina la divergencia, pero añade sincronización.

7.7 Precisión: `double` frente a `float`

Qué es. Hacer el cálculo en 32 bits.

Por qué lo usamos. En GPUs GeForce el FP64 es mucho más lento que el FP32 **[por verificar]**, y `float` reduce a la mitad los bytes de `vX` y `vContrib`.

Cómo se usa. Implementa el kernel con un `typedef` de tipo real y compara tiempo y error frente al oráculo (experimento E10). Con $u_{\text{float}} \approx 6 \cdot 10^{-8}$ el error relativo esperado es del orden de 10^{-6}: hay que relajar la tolerancia y justificarlo.

Alternativas. Mixta: productos en `float` y acumulación en `double`. La validación de corrección del enunciado debe hacerse con `double`.

7.8 Streams y solapamiento

Qué es. Colas independientes que permiten solapar copias y kernels (requiere memoria anclada).

Por qué lo usamos. Si se evalúan muchos puntos o ecuaciones, la copia del lote $i+1$ se solapa con el cálculo del lote i.

Cómo se usa. `cudaStreamCreate`, `cudaMemcpyAsync(..., stream)` y `kernel<<<b, t, 0, stream>>>`. Compruébalo en la línea temporal de Nsight Systems.

Alternativas. Para una sola evaluación no aporta nada: es una optimización para el modo por lotes (E11).

8 Sistemas de ecuaciones y Jacobiano

Tema 8: contenidos que necesitas para las fases F10 (apartado 2.1.3, 20 %).

8.1 Matriz Jacobiana

Qué es. Para un sistema $F_i(\mathbf{x}) = \text{lhs}_i(\mathbf{x}) - \text{rhs}_i = 0$, $i = 1..K$, el Jacobiano es la matriz $K \times N$ con $J_{ij} = \partial F_i / \partial x_j$. Cada fila es el gradiente de una ecuación, y la constante rhs_i no influye.

Por qué lo usamos. Es la extensión pedida. Es la pieza que usaría el método de Newton para resolver el sistema: $J\,\Delta\mathbf{x} = -F(\mathbf{x})$ (no pedido, pero buen comentario para la memoria).

Cómo se usa. Sistema del enunciado, ya corregido (`3x^2 + yz + z + 5 = 2`, `2x + yz^3 + 2x + 2 = 0`, `3xz + 8yz + z^2 + 1 = 4`). En $(1, 2, 3)$, salida de `jac.c`:

$$F = (15,\ 60,\ 63), \qquad J = \begin{pmatrix} 6 & 3 & 3 \\ 4 & 27 & 54 \\ 9 & 24 & 25 \end{pmatrix}.$$

Por ejemplo, $J_{23} = \partial(yz^3)/\partial z = 3yz^2 = 54$ y $J_{33} = 3x + 8y + 2z = 25$. Estos valores son el caso de prueba obligatorio de F10.

Alternativas. Ninguna para la definición.

8.2 Estructura de datos para sistemas

Qué es. Concatenar las K ecuaciones en un solo CSR global y añadir `vOffEc[K+1]` (primer monomio de cada ecuación) y `vRhs[K]`.

Por qué lo usamos. Un único CSR permite reutilizar los mismos kernels y bucles: un monomio solo necesita saber a qué fila del Jacobiano contribuye. El enunciado pide expresamente *adaptar las estructuras de datos*.

Cómo se usa. El índice de destino pasa de `v` a `ec * nVars + v`. En GPU, la ecuación de cada monomio se obtiene con una búsqueda binaria en `vOffEc` o, más barato, guardando `vIdEc[M]` (4 bytes por monomio).

Alternativas. Un array de K estructuras `Poli` independientes (lo que hace el prototipo `jac.c`): más simple, pero obliga a K lanzamientos de kernel o a regiones OpenMP por ecuación, con mal equilibrio si las ecuaciones tienen tamaños distintos.

8.3 Paralelización del Jacobiano

Qué es. Tres niveles de paralelismo: ecuaciones, monomios dentro de la ecuación y variables.

Por qué lo usamos. Determina la escalabilidad del apartado y enlaza con el estudio de anidado.

Cómo se usa.

- OpenMP: un solo `parallel for` sobre todos los monomios del CSR global (equilibrado de forma natural), con la variante de dos fases y una CSC por pares `(ec, v)`. O bien anidado: externo por ecuaciones e interno por monomios, que compara con lo anterior en el estudio.
- CUDA: el mismo `kContrib` sobre el CSR global; en la fase 2, un hilo o warp por entrada no nula de J.

Alternativas. Paralelizar solo por ecuaciones: trivial y sin conflictos, pero con K pequeño (3 en el enunciado) no hay paralelismo suficiente.

8.4 Almacenamiento del Jacobiano

Qué es. Denso ($8KN$ bytes) o disperso (solo las entradas no nulas, en CSR por filas).

Por qué lo usamos. Con $K = N = 10^3$ el denso ocupa 8 MB (bien); con $K = N = 10^4$, 800 MB (límite en la GPU); con $K = N = 10^5$, 80 GB (imposible). El número de no nulos está acotado por F.

Cómo se usa. Usa denso hasta unos 100 MB y disperso por encima. Para el disperso, la CSC por pares `(ec, v)` de la fase 2 *es* ya la estructura del Jacobiano: las posiciones no nulas son las claves únicas.

Alternativas. Devolver solo $J \cdot \mathbf{v}$ (producto Jacobiano por vector), como hacen las bibliotecas de AD: evita guardar J, pero no es lo que pide el enunciado.

9 Validación y benchmarking

Tema 9: contenidos que necesitas para las fases F2, F6, F11 y F15.

9.1 Pirámide de pruebas

Qué es. Tres niveles: casos a mano (ejemplo del enunciado, sistema del enunciado y casos límite), casos aleatorios contra oráculo y comparación cruzada entre versiones (C, C++, Rust, OpenMP y CUDA).

Por qué lo usamos. El requisito eliminatorio de CUDA obliga a demostrar la corrección, y en la entrevista te pueden pedir que lo enseñes en directo.

Cómo se usa. Casos límite que debe incluir `tests/casos/`:

- Función constante (`5`): gradiente nulo.
- Variable que no aparece en ningún monomio: su derivada es 0.
- Exponente 1 y exponente alto (x^5 en $x = 1.5$).
- Punto con $x_j = 0$ (detecta el método de división) y con coordenadas negativas.
- Coeficientes negativos y cancelación ($x - x$).
- Monomios repetidos y variable repetida en un monomio (`xx`).
- $M = 1$, $N = 1$, y M no múltiplo del tamaño de bloque (comprueba el `if (m >= nMon)`).

Alternativas. Pruebas basadas en propiedades (*property-based*, con Hypothesis en Python): generan casos al azar y reducen el contraejemplo mínimo. Muy potentes para el parser; añádelas si sobra tiempo.

9.2 Metodología de medida

Qué es. El protocolo para que los tiempos sean fiables y comparables (Hoefler y Belli, 2015).

Por qué lo usamos. El análisis crítico que se evalúa solo es defendible si las medidas lo son.

Cómo se usa.

1. Máquina sin otras cargas (comprueba con `uptime` y `nvidia-smi`); anota fecha, hora y nodo.
2. Una ejecución de calentamiento no medida y después $R = 10$ repeticiones.
3. Reporta mediana y desviación típica (o intervalo intercuartílico). La mediana es robusta a interferencias puntuales.
4. Afinidad fija (`OMP_PROC_BIND`, `OMP_PLACES`).
5. Mismo binario y mismas opciones para todas las variantes; el secuencial de referencia, compilado igual.
6. Tiempos separados: lectura, parseo, construcción de CSC, cálculo, H2D, kernel, D2H.
7. Verificación de cada resultado medido (que la optimización no haya roto nada).

Alternativas. Media e intervalo de confianza al 95 %: también correcto si la distribución es estable, pero más sensible a valores atípicos.

9.3 Gráficas y tablas

Qué es. Las representaciones de los resultados para la memoria.

Por qué lo usamos. Una gráfica bien elegida hace evidente la conclusión.

Cómo se usa. Gráficas mínimas: tiempo y speedup frente a hilos (con la línea ideal y la de Amdahl), eficiencia frente a hilos, barras apiladas H2D/kernel/D2H por tamaño de problema, tiempo de kernel frente a tamaño de bloque y mapa de calor para el anidado ($h_{\text{ext}} \times h_{\text{int}}$). El prototipo `graficas.py` calcula mediana, desviación, speedup y eficiencia con Polars y dibuja con Matplotlib. Figura 1: salida del prototipo en la máquina de nube (solo demuestra que la cadena funciona).

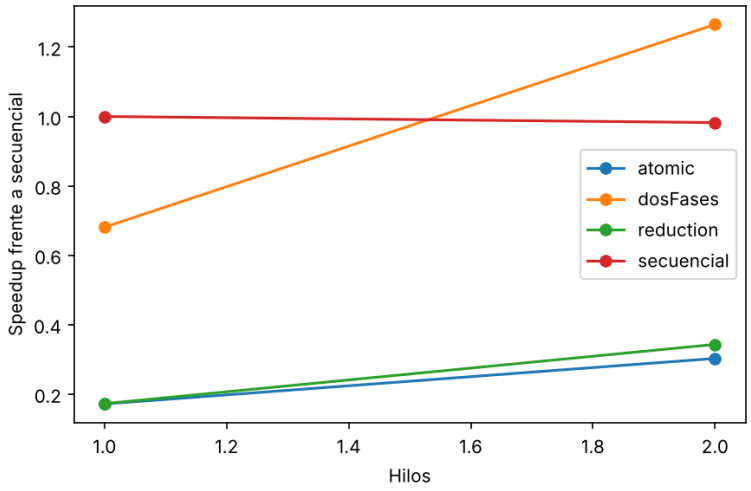


Figura 1: Speedup orientativo con 1 y 2 hilos (nube de 2 vCPU, $N = 10^3$, $M = 10^5$). No usar en la memoria.

Alternativas. Gnuplot, o pgfplots dentro de LaTeX (mejor integración tipográfica, más lento de iterar).

10 Herramientas y entorno

Tema 10: contenidos que necesitas para las fases F0, F12 y F15.

10.1 Git e identidad de autor

Qué es. Control de versiones. Cada commit guarda autor y *committer*.

Por qué lo usamos. Quieres que solo figures tú en el repositorio, y el enunciado valora mostrar la evolución incremental del código.

Cómo se usa. Configura una vez `git config user.name "Antonio Millán Rodríguez"` y `git config user.email "antoniomillanr45@gmail.com"`. Un commit por fase, con un mensaje que nombre la fase, y una etiqueta por punto de control (`git tag pc1`). Verifica la autoría con:

```
git log --format='%an <%ae> | %cn <%ce>'
git log --format=%B | grep -iE "co-authored|claude"
```

El segundo comando no debe devolver nada. La rama `claude/fervent-bell-1ekfj8` tiene un commit con autor *Claude*: si la fusionas, rehaz ese commit con tu identidad (`git cherry-pick` y `git commit --amend --reset-author`) en lugar de fusionarlo tal cual, y borra la rama remota.

Alternativas. Ramas por fase con *pull requests*: más ordenado, pero excesivo para un proyecto individual.

10.2 Make

Qué es. Herramienta que recompila solo lo que ha cambiado según reglas de dependencias.

Por qué lo usamos. Un `make all` y un `make test` son lo que el profesor ejecutará en el servidor.

Cómo se usa. Ver `docs/guia/codigo/Makefile` (probado): objetivos `all`, `grad`, `test` y `clean`, con `ARCH = -arch=sm_89` como variable que se cambia en el servidor (`make grad ARCH=-arch=sm_80`).

Alternativas. CMake: detecta CUDA y OpenMP de forma portable, pero añade una capa de complejidad innecesaria para un proyecto de 10 ficheros.

10.3 Contenedores: Docker y Apptainer

Qué es. Un contenedor empaqueta el sistema, las dependencias y el binario. Docker es el estándar general; Apptainer (antes Singularity; Kurtzer et al., 2017) es el habitual en servidores HPC porque no exige privilegios de `root`.

Por qué lo usamos. El enunciado pide *incluir la solución en un sistema de contenedores*, y sirve para fijar versiones (reproducibilidad).

Cómo se usa. Dockerfile orientativo (`docs/guia/codigo/Dockerfile`; no se ha podido construir en este entorno; etiqueta de imagen y compatibilidad de versiones con Ubuntu 22.04 **[por verificar]**):

```
FROM nvidia/cuda:12.4.1-devel-ubuntu22.04
RUN apt-get update && apt-get install -y --no-install-recommends make python3 python3-pip &&
    rm -rf /var/lib/apt/lists/*
RUN pip3 install --no-cache-dir sympy==1.14.0 numpy==2.5.3 polars==2.1.0 matplotlib==3.11.2
WORKDIR /app
COPY . .
RUN make all grad
CMD ["make", "test"]
```

Ejecución con GPU: `docker run --gpus all imagen`; requiere NVIDIA Container Toolkit en el anfitrión. Con Apptainer: `apptainer build grad.sif docker-daemon://imagen:latest` y `apptainer run --nv grad.sif`.

Alternativas. Podman (Docker sin demonio y sin `root`) o un entorno `conda`/`venv` más un `Makefile`: no es un contenedor y no cumple la tarea 6.

Trampa habitual

La versión de CUDA de la imagen no puede ser más nueva de lo que admite el driver del servidor. Mira `nvidia-smi` (“CUDA Version”) antes de elegir la etiqueta.

10.4 Librerías de Python

Librería	Papel	Por qué sí o por qué no como base
SymPy 1.14	Validación (oráculo)	Exacta, pero lenta para $M > 10^4$: solo casos pequeños
NumPy 2.5	Generación	Generación vectorizada de enteros aleatorios con `default_rng`
Polars 2.1	Generación y análisis	Agregaciones rápidas de CSV de tiempos; Pandas también valdría
Matplotlib 3.11	Análisis	Gráficas para la memoria
JAX o PyTorch	Comparación opcional	AD de referencia del estado del arte; no son base porque el enunciado pide C

Versiones probadas en esta guía; fíjalas en `requirements.txt`.

10.5 Trabajo en el servidor de prácticas

Qué es. El entorno donde se harán las pruebas finales.

Por qué lo usamos. Es obligatorio, y su hardware determina todos los resultados.

Cómo se usa. Copia con `git clone` (o `rsync`), compila allí con `ARCH` adaptado y lanza los benchmarks con `nohup ./reproduce.sh &` o dentro de `tmux`. Si hay gestor de colas (Slurm), usa `sbatch` y pide los nodos en exclusiva **[por verificar]**.

Alternativas. Medir en el portátil sirve para desarrollar y como segundo entorno comparativo en la memoria, pero no sustituye al servidor.

11 IA generativa, prompts y memoria

Tema 11: contenidos que necesitas para las fases F13 y F14 (apartados 2.1.4 y 2.1.5, 30 % en total).

11.1 Ingeniería de prompts

Qué es. Diseñar la petición a un modelo de lenguaje para obtener una respuesta útil: contexto, objetivo, restricciones, formato de salida y ejemplos.

Por qué lo usamos. Se puntúa *la calidad y el contexto* de los prompts y su refinamiento incremental.

Cómo se usa. Estructura de prompt que funciona para código:

1. Rol y contexto: asignatura, enunciado y estado actual del código (adjunto).
2. Tarea concreta y criterio de éxito verificable (“debe dar (8, 15, 8)”).
3. Restricciones: C11, OpenMP 4.5, sin comentarios, nomenclatura, sin dependencias.
4. Formato: solo el fichero, o un diff.
5. Iteración: pega el error de compilación o los tiempos medidos y pide un diagnóstico, no solo un arreglo.

Ejemplo de refinamiento: “paraleliza el gradiente con OpenMP” → “paraleliza `gradSec` sobre CSR con OpenMP evitando atómicos; propone dos variantes y explica su coste en memoria” → “la variante *reduction* es 3 veces más lenta que el secuencial con 2 hilos; aquí están los tiempos; ¿por qué?”.

Alternativas. Usar la IA solo para depurar o documentar (más seguro, menos material para el apartado). Lo valorado es mostrar el proceso completo, incluidas las respuestas erróneas y cómo las detectaste.

11.2 Registro de interacciones

Qué es. Un diario en `docs/prompts.md`.

Por qué lo usamos. Reconstruirlo al final es costoso y poco creíble. El enunciado pide, para cada IA: ámbito de uso, lista de prompts, soluciones obtenidas y valoración final.

Cómo se usa. Plantilla por entrada:

```
## Fecha  IA y modelo  Fase
Prompt: ...
Respuesta resumida: ...
Fragmento clave: ...
Problema detectado y cómo: ...
Commit resultante: ...
```

Incluye esta guía como una interacción (ámbito: planificación y estudio).

Alternativas. Exportar las conversaciones completas como anexo: útil como respaldo, pero el cuerpo de la memoria debe ser un análisis, no un volcado.

11.3 Memoria con el esquema oficial

Qué es. El documento de entrega: Abstract, Introducción, El problema, Paralelización (secuencial, multinúcleo, GPU, extensiones), Resultados (entorno, resultados), Conclusiones y Bibliografía.

Por qué lo usamos. Vale un 15 % y es lo que se lee antes de la entrevista.

Cómo se usa. Cada tema de esta guía alimenta una sección: 1 y 2 van a "El problema"; 3, 5, 6, 7 y 8, a "Paralelización"; 4 y 9, a "Resultados"; 12, a "Introducción". Escribe cada sección al cerrar su fase, no al final. Cita solo referencias que hayas consultado.

Alternativas. LaTeX (recomendado: fórmulas y bibliografía) o Markdown con Pandoc (más rápido, menos control).

12 Estado del arte

Tema 12: contenidos que necesitas para las fases F14 (introducción) y contexto de todas.

12.1 Diferenciación automática

Qué es. La técnica estándar para gradientes en aprendizaje automático y optimización (Baydin et al., 2018; Griewank y Walther, 2008). El modo inverso calcula el gradiente completo con un coste acotado por un pequeño múltiplo constante del coste de evaluar f (principio del gradiente barato).

Por qué lo usamos. Sitúa el proyecto: nuestro algoritmo prefijo-sufijo es un modo inverso especializado para productos, sin cinta (*tape*) ni grafo, gracias a conocer la estructura polinómica.

Cómo se usa. Herramientas: PyTorch (`autograd`; Paszke et al., 2019), JAX (`grad`, `jit`, `vmap`; Bradbury et al., 2018), Enzyme (AD a nivel de LLVM, incluso de kernels CUDA; Moses y Churavy,

2020) y CasADi. Un experimento opcional valioso: comparar tu CUDA con `jax.grad` sobre el mismo polinomio en la misma GPU.

Alternativas. — (es el contexto, no una opción de implementación).

12.2 Evaluación de polinomios y sus derivadas en GPU

Qué es. Existe trabajo específico sobre evaluar y diferenciar sistemas polinómicos en GPU para métodos de homotopía, por ejemplo Verschelde y Yoffe (2012) **[por verificar]**. Usan, entre otras ideas, productos tipo Speelpenning para los monomios.

Por qué lo usamos. Confirma que el planteamiento (monomio por hilo y productos prefijo y sufijo) es el adecuado y da una referencia seria para la introducción.

Cómo se usa. Lee el resumen y la sección de algoritmo antes de citarlo. Si no puedes acceder, no lo cites.

Alternativas. SpMV en GPU (Bell y Garland, 2009) es el problema hermano: mismo patrón disperso, mismas técnicas (CSR, ELL, *sliced* ELL). Sus conclusiones sobre formatos son directamente aplicables.

12.3 En qué se diferencia este proyecto

- Derivación exacta especializada en polinomios dispersos, sin grafo de AD: menos memoria y sin dependencias.
- Comparación sistemática de estrategias de acumulación (*reduction*, *atomic*, dos fases) en CPU y GPU sobre el mismo problema.
- Extensión al Jacobiano con un CSR global y estudio de anidado.
- Documentación del uso de IA generativa como parte del método.

No es más rápido que una biblioteca de AD en general: es más rápido *en este caso particular*, y eso es lo que hay que demostrar con medidas.

13 Tabla de decisiones

Id	Decisión	Opciones	Recomendación	Motivo y qué medir para decidir	Fase
D1	Lenguaje de las versiones paralelas	C, C++	C (tomada)	Integración directa con OpenMP y CUDA; CUDA se compila como C++ con estilo C	F0
D2	Nombres de ficheros	`.cpp` actuales, `.c`/`.cu`	`.c` y `.cu` en `src/`	`nvcc` necesita `.cu`; coherencia con D1	F0
D3	Sistema de compilación	Make, CMake	Make	Pocos ficheros; el profesor solo ejecuta `make`	F0
D4	Formato de validación	Texto del enunciado, CSV, binario	Texto con la regla letra+dígitos	Notación del enunciado y legible; base del oráculo	F1
D5	Formato de benchmark	Texto, CSV, binario	Binario si el parseo pesa	Medir el tiempo de parseo frente al de cálculo en el caso M: si el parseo supera el 10 % del total, binario	F1
D6	Rangos del generador	Los actuales ($e \le 100$, $k \le 100$), seguros	$c \le 9$, $e \le 5$, $k \le 16$, $x \in [0.5, 1.5]$	Evita `inf`; comprobar que máx $\lvert g_j \rvert < 10^{300}$	F1
D7	Representación	Densa, COO, CSR, ELL	CSR (cambia la decisión densa del 18-oct)	Memoria $O(F)$; medir la densidad $F/(MN)$ de tus casos	F3
D8	Algoritmo por monomio	Ingenuo, división, prefijo-sufijo	Prefijo-sufijo	$O(k)$ sin divisiones; medida: el ingenuo es 3 veces más lento	F4
D9	Potencias	`pow`, bucle, cuadrados, tabla	Cuadrados; tabla en F9 si se mide mejora	Medir tiempo por factor con cada una en el caso M	F4, F9
D10	Precisión	`double`, `float`, mixta	`double`; `float` como experimento	Corrección exigida; medir tiempo y error (E10)	F4, F9
D11	Tolerancia	Absoluta, relativa, mixta	$\le 10^{-12}$ máx$(1, \lvert g \rvert)$	Coeficientes positivos y $x \in [0.5, 1.5]$; relajar con justificación si hay cancelación	F6
D12	Acumulación en OpenMP	`critical`, `atomic`, `reduction`, privatizada, dos fases	Dos fases como principal; las demás como comparación	Determinista y sin conflictos; medir las cinco frente a N (E3, E8)	F7
D13	`schedule`	static, dynamic, guided, por factores	Decidir con datos; partir de dynamic,256	Barrido E5; mirar la varianza entre hilos P y E	F7
D14	Anidado	Anidado real, `collapse`, `taskloop`	Estudiar las tres	El enunciado pide el estudio; medir E6	F7
D15	Mapeo CUDA	Hilo por monomio, por factor, warp por monomio, dos fases	Hilo por monomio con atómicos (base) y dos fases (optimizada)	Corrección primero; comparar en E7 y E8	F8, F9
D16	Tamaño de bloque	32 a 1024	Elegir por barrido	Medir E7 y contrastar con la ocupación teórica	F8
D17	Memoria para `vX`	Global con `__ldg`, compartida, constante	`__ldg`; compartida si $N \le 6144$ y mejora	Constante descartada por los accesos divergentes; medir con `ncu`	F9
D18	Layout en GPU	CSR, ELL transpuesto, sliced ELL ordenado	CSR; probar sliced ELL	Medir el ancho de banda efectivo y la eficiencia de warp	F9

Id	Decisión	Opciones	Recomendación	Motivo y qué medir para decidir	Fase
D19	Estructura del sistema	K `Poli` separados, CSR global	CSR global con `vIdEc`	Un solo bucle o kernel equilibrado	F10
D20	Almacenamiento de J	Denso, disperso	Denso hasta unos 100 MB	$8KN$ bytes; disperso por encima	F10
D21	Contenedor	Docker, Apptainer, Podman	Según el servidor	Preguntar si hay Docker con NVIDIA Container Toolkit o Apptainer y si hay `root`	F12
D22	Paralelismo en Rust	Ninguno, rayon	Ninguno	Fuera del enunciado; solo secuencial	F5
D23	Qué tiempos se reportan	Solo cálculo, total	Ambos, separados	El enunciado pide el desglose; Amdahl exige conocer la parte secuencial	F11

14 Experimentos, riesgos y calendario

14.1 Experimentos planificados

Casos de tamaño (definidos en el concepto 2.3): S ($N = 10^3$, $M = 10^4$), M ($N = 10^4$, $M = 10^6$) y L ($N = 10^5$, $M = 10^7$), todos con $k \in [1, 16]$, $e \in [1, 5]$, $c \in [1, 9]$, $x \in [0.5, 1.5]$, semilla de función 0 y semilla de punto 1. Hilos: $\{1, 2, 4, 6, 8, 12\}$ en el portátil y potencias de 2 hasta el número de núcleos en el servidor. $R = 10$ repeticiones y mediana.

Id	Objetivo	Configuración	Resultado esperado
E1	Corrección	Ejemplo del enunciado; sistema del enunciado; oráculo ($N = 50$, $M = 200$, semilla 7; $N = 300$, $M = 2000$, semilla 11); casos límite del concepto 9.1	$(8, 15, 8)$; filas $(6, 3, 3)$, $(4, 27, 54)$, $(9, 24, 25)$; error 0 frente al oráculo (ya obtenido con el prototipo)
E2	Coste secuencial	C, C++ y Rust en S, M y L	Tiempo lineal en F (unos 16 ns por factor en la nube); diferencias entre lenguajes por debajo del 20 % **[por verificar]**
E3	Escalado fuerte OpenMP	Caso M, cinco variantes de D12, todos los hilos	Dos fases con mejor eficiencia; *reduction* empeora al crecer los hilos por la fusión $O(pN)$; techo por ancho de banda
E4	Escalado débil OpenMP	$M = 10^5 \cdot p$, $N = 10^4$	Tiempo casi constante hasta saturar el ancho de banda
E5	Planificación	Caso M, 8 hilos, static, dynamic y guided con trozos $\{16, 64, 256, 1024, 4096\}$	dynamic o guided por encima de static en el portátil (núcleos P y E); trozo pequeño penalizado
E6	Anidado	$P \in \{1, 16, 256\}$ puntos, caso S, todas las combinaciones $h_{\text{ext}} \times h_{\text{int}}$	Con $P = 256$ gana el externo o `collapse`; con $P = 1$ solo sirve el interno
E7	CUDA: bloque y desglose	Casos S, M y L; bloques de 32 a 1024; `kMonoAtom` y dos fases	H2D mayor que el kernel en una evaluación; 32 hilos por bloque peor; meseta entre 128 y 512
E8	Contención de atómicos	$F = 8.5 \cdot 10^6$ fijo y $N \in \{10, 10^3, 10^5\}$; CPU y GPU	Atómicos muy lentos con $N = 10$; dos fases insensible a N
E9	Jacobiano	$K \in \{10, 100, 1000\}$, $N = 10^3$, $M = 10^3$ por ecuación	CSR global equilibrado; denso hasta $K = 1000$ (8 MB)
E10	`float` frente a `double`	Caso M en GPU	`float` más rápido en GeForce; error relativo del orden de 10^{-6}
E11	Lotes de puntos en GPU	Caso M con 1, 10 y 100 puntos y estructura residente	El coste de H2D se amortiza; speedup frente al secuencial creciente con los puntos

14.2 Riesgos

Riesgo	Impacto	Mitigación
CUDA no coincide con el secuencial (eliminatorio)	Muy alto	Validar contra el oráculo en cada commit; casos límite; comprobar errores CUDA; comparar con tolerancia, no con `==`
Servidor sin GPU disponible, ocupado o con otro modelo	Alto	Medir pronto (PC3); tener resultados del portátil como respaldo y declararlo; `ARCH` parametrizado
Driver del servidor incompatible con la versión de CUDA o del contenedor	Medio	`nvidia-smi` y `nvcc --version` la primera semana; elegir la imagen según el driver
Docker no permitido en el servidor	Medio	Apptainer con `--nv`; si nada funciona, documentarlo y entregar el Dockerfile probado en el portátil
Desbordamientos o `inf` en casos grandes	Medio	Rangos de D6; comprobar `isfinite` en los tests
El parseo domina el tiempo y oculta el speedup	Medio	Medir por separado; formato binario (D5)
Medidas ruidosas por compartir el servidor	Medio	Mediana de 10, afinidad, horas valle, anotar la carga
Retraso en la parte de IA y memoria (30 %)	Alto	Registrar prompts desde ya; escribir cada sección al cerrar su fase
Fecha de entrega desconocida	Alto	Confirmarla esta semana en el aula virtual; ajustar el calendario
Ambigüedades del enunciado (pesos, formato, contenedores)	Medio	Preguntar al profesor por escrito y citar la respuesta en la memoria
Pérdida de trabajo	Medio	Push diario; etiquetas por punto de control

14.3 Calendario con puntos de control

Supuesto: entrega opcional de OpenMP hacia el 7-dic-2026 y entrega final hacia el 25-ene-2027. Ajusta las fechas cuando confirmes las reales.

Semana	Fases	Punto de control: qué debe estar listo	Si no lo está
2–8 nov	F0, F1	Repo reorganizado, generadores con semilla, entorno del servidor guardado	Usar el `generador.py` existente con los rangos de D6
9–15 nov	F2, F3, F4	**PC1**: secuencial en C validado con el ejemplo y el oráculo	Usar los prototipos de esta guía tal cual y seguir
16–22 nov	F5, F6	C++ y Rust; `make test` en verde	Posponer Rust: no puntúa en el enunciado
23–29 nov	F7	Cinco variantes OpenMP correctas	Quedarse con *reduction* y dos fases
30 nov–6 dic	F7, F11 (CPU)	**PC2**: escalabilidad, schedule y anidado medidos en el servidor; entrega opcional	Entregar con lo medido; el anidado puede ir después
7–13 dic	F8	**PC3**: CUDA base correcto con desglose de tiempos en el servidor	Prioridad máxima: es eliminatorio. Pedir ayuda en tutoría
14–20 dic	F10, F9	Jacobiano en secuencial, OpenMP y CUDA; primera optimización CUDA	F9 es opcional: saltarla antes que F10
21 dic–10 ene	F11, F12	**PC4**: todos los benchmarks en CSV y figuras; contenedor funcionando	Reducir la rejilla de experimentos a S y M
11–17 ene	F13, F14	**PC5**: memoria completa en borrador y registro de IA cerrado	Recortar texto, no resultados

Semana	Fases	Punto de control: qué debe estar listo	Si no lo está
18–24 ene	F15	`reproduce.sh` regenera todo; revisión final; ensayo de la entrevista	Entregar lo que hay: el contenido mínimo ya existe desde PC4

15 Validación y reproducibilidad

Tests. `make test` ejecuta: el ejemplo (`grad = (8, 15, 8)`), el sistema (fila `(9, 24, 25)`), los casos límite del concepto 9.1 y dos casos aleatorios contra el oráculo. Cada versión paralela se compara después con el secuencial en el caso M. El prototipo ya incluye los dos primeros en el `Makefile`.

Tolerancias.

Comparación	Tolerancia	Motivo
Secuencial frente a oráculo (puntos enteros)	$10^{-12}\,\text{máx}(1, \lvert g\rvert)$	Error obtenido: 0
Paralelo `double` frente a secuencial	$10^{-12}\,\text{máx}(1, \lvert g\rvert)$	Solo cambia el orden de las sumas (obtenido $\leq 5.4 \cdot 10^{-15}$)
GPU `float` frente a secuencial	$10^{-4}\,\text{máx}(1, \lvert g\rvert)$	$u_{\text{float}} \approx 6 \cdot 10^{-8}$ por unos 10^3 términos

Semillas. Función: 0; punto: 1; casos de oráculo: 7 y 11. Se escriben en la cabecera de cada fichero generado y en cada fila del CSV de tiempos.

Versiones fijadas. Las de la tabla 1, en `requirements.txt` y en el Dockerfile. `reproduce.sh` guarda las del servidor en `resultados/entorno.txt`.

Herramienta	Versión probada en la guía	Servidor
gcc	13.3.0 (OpenMP 4.5 o superior)	Por medir
nvcc	12.0.140 (solo compilación)	Por medir
rustc	1.97.0	Por medir
Python	3.13	Por medir
SymPy, NumPy, Polars, Matplotlib	1.14.0, 2.5.3, 2.1.0, 3.11.2	Fijar en `requirements.txt`

Tabla 1: Versiones.

Metodología de medida. La del concepto 9.2: calentamiento, $R = 10$, mediana y desviación, afinidad, tiempos separados por etapa y verificación de cada ejecución medida.

Script único. `reproduce.sh` (probado en la parte de CPU): guarda el entorno, compila, pasa los tests y el oráculo, ejecuta los benchmarks, genera las gráficas y, si hay GPU, barre los tamaños de bloque.

```
make all
make test
for caso in "50 200 7" "300 2000 11"; do
  python3 oraculo.py $caso
  gcc -O2 -o compara compara.c poli.c -lm
  ./compara
done
./bench.sh 1000 100000 5 resultados/tiempos.csv
python3 graficas.py resultados/tiempos.csv
if command -v nvidia-smi > /dev/null; then
```

```
  make grad
  for b in 32 64 128 256 512 1024; do ./grad 10000 1000000 "$b"; done > resultados/cuda.txt
fi
```

Al llevarlo al repositorio final, amplía `bench.sh` a la rejilla de la sección 14.

16 Orientación a la entrega académica

El objetivo externo es la entrega de la práctica y, en tu caso, una nota de 9 o más para optar a matrícula de honor. Cómo orientar el trabajo:

- **Asegura lo eliminatorio primero**: CUDA correcto (PC3) vale más que cualquier optimización.
- **Rúbrica implícita**: cada requisito del enunciado debe tener una tabla o gráfica con nombre propio en la memoria: `omp_get_wtime`, escalabilidad, anidado, desglose CUDA, tamaños de bloque, Jacobiano, contenedor, prompts.
- **Análisis crítico**: para cada gráfica, una explicación causal (Amdahl, ancho de banda, contención, desequilibrio) apoyada en una medida adicional (`perf`, `ncu`, Karp-Flatt). Es lo que separa un 7 de un 9.
- **Parte opcional de CUDA**: con dos optimizaciones bien medidas (dos fases y ordenación por longitud, por ejemplo) y una descartada con datos basta para destacar.
- **IA (15 %)**: muestra la evolución, los errores de la IA que detectaste (por ejemplo, un `reduction` ingenuo $O(k^2)$ o un `atomicAdd` con CAS obsoleto) y cómo afinaste los prompts.
- **Entrevista**: debes poder explicar sin notas la fórmula del gradiente de un monomio, por qué CSR, por qué dos fases, qué mide cada evento CUDA y qué limita el speedup. Los temas 1, 2, 5 y 6 cubren esas preguntas.
- **Bibliografía**: cita las referencias relevantes del enunciado (Chandra; Mattson et al.; Brownlee; OpenAI) y las de esta guía que hayas leído.

Referencias

[1] Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Spring Joint Computer Conference*, 483–485.

[2] Baydin, A. G., Pearlmutter, B. A., Radul, A. A. y Siskind, J. M. (2018). Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18(153), 1–43.

[3] Bell, N. y Garland, M. (2009). Implementing sparse matrix-vector multiplication on throughput-oriented processors. *Proceedings of SC'09*.

[4] Bradbury, J. et al. (2018). JAX: composable transformations of Python+NumPy programs. https://github.com/jax-ml/jax.

[5] Brownlee, J. (2023). *Prompt Engineering for Coders*. Machine Learning Mastery (citada en el enunciado).

[6] Chandra, R. et al. (2001). *Parallel Programming in OpenMP*. Morgan Kaufmann.

[7] Goldberg, D. (1991). What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1), 5–48.

[8] Griewank, A. y Walther, A. (2008). *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation* (2.ª ed.). SIAM.

[9] Gustafson, J. L. (1988). Reevaluating Amdahl's law. *Communications of the ACM*, 31(5), 532–533.

[10] Higham, N. J. (2002). *Accuracy and Stability of Numerical Algorithms* (2.ª ed.). SIAM.

[11] Hoefler, T. y Belli, R. (2015). Scientific benchmarking of parallel computing systems. *Proceedings of SC'15*.

[12] Hwu, W. W., Kirk, D. B. y El Hajj, I. (2022). *Programming Massively Parallel Processors* (4.ª ed.). Morgan Kaufmann.

[13] Knuth, D. E. (1997). *The Art of Computer Programming, vol. 2: Seminumerical Algorithms* (3.ª ed.), sección 4.6.3. Addison-Wesley.

[14] Kurtzer, G. M., Sochat, V. y Bauer, M. W. (2017). Singularity: scientific containers for mobility of compute. *PLoS ONE*, 12(5), e0177459.

[15] Mattson, T. G., Sanders, B. A. y Massingill, B. L. (2004). *Patterns for Parallel Programming*. Addison-Wesley.

[16] Meurer, A. et al. (2017). SymPy: symbolic computing in Python. *PeerJ Computer Science*, 3, e103.

[17] Moses, W. y Churavy, V. (2020). Instead of rewriting foreign code for machine learning, automatically synthesize fast gradients. *NeurIPS 2020*.

[18] NVIDIA. *CUDA C++ Programming Guide* y *CUDA C++ Best Practices Guide*. `https://docs.nvidia.com/cuda/` (versión según el toolkit del servidor).

[19] OpenAI (2023). Best practices for prompt engineering with AI models (citada en el enunciado). `https://platform.openai.com/docs/guides/gpt-best-practices`.

[20] OpenMP Architecture Review Board (2021). *OpenMP Application Programming Interface, Version 5.2*. `https://www.openmp.org/specifications/`.

[21] Paszke, A. et al. (2019). PyTorch: an imperative style, high-performance deep learning library. *NeurIPS 2019*.

[22] Saad, Y. (2003). *Iterative Methods for Sparse Linear Systems* (2.ª ed.). SIAM.

[23] Verschelde, J. y Yoffe, G. (2012). Evaluating polynomials in several variables and their derivatives on a GPU computing processor. *IEEE IPDPS Workshops* **[por verificar]**.

[24] Williams, S., Waterman, A. y Patterson, D. (2009). Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4), 65–76.